CRE Diagnostic & Repair Framework

Constraint–Reward–Environment Inconsistency Detection, Scoring, and Language-Guided Repair for Reinforcement Learning

Document Part	*Content*	Format
Part I	*NeurIPS-Standard Method Section*	Research paper chapter
Part II	*System Development Handbook*	Agent implementation spec
Appendix A	*Notation Reference Table*	Symbol table
Appendix B	*Full Prompt Templates*	LLM-α–LLM-ε
Appendix C	*JSON Schema Definitions*	All 6 record types
Appendix D	*Worked End-to-End Example*	Autonomous overtaking

Version 1.0 | April 10, 2026

Usage. Part I is structured for direct inclusion as the `Method` section of a NeurIPS submission (8 sections, self-contained theorems and proofs). **Part II** is a complete agent-oriented developer specification: every module functional contract, data schema, algorithm, configuration parameter, prompt template, and test case is specified to the level of autonomous implementation without further human clarification. All cross-references between parts use consistent numbering.

Contents

I NeurIPS-Standard Method Section 4

1 Problem Formulation and Notation 4
- 1.1 Specification Triple and Formal Representation 4
- 1.2 Three-Class Inconsistency Taxonomy 5
- 1.3 Formal Problem Statement: Detection, Localization, and Repair 6

2 Stage I — Specification Parsing and Representation 7
- 2.1 LLM-α: Parse and Ambiguity Detection 7
- 2.2 Structured Specification Builder 7
- 2.3 Internal Consistency Pre-Check 8

3 Stage II — Two-Axis Estimation Framework 8
- 3.1 Axis Definitions and Theoretical Motivation 8
- 3.2 Canonical Reporter Selection 8
- 3.3 Primary Estimators — Constraint–Reward Dimension 9
- 3.4 Primary Estimators — Environment–Constraint Dimension 10
  - 3.4.1 E–C Aggregation Mode 10
- 3.5 Primary Estimators — Environment–Reward Dimension 11
- 3.6 Utility Oracle Assumptions 11
- 3.7 Non-Degeneracy Conventions for Ratio and Correlation Reporters 12
- 3.8 Enhanced Estimators — Mechanistic Layer 13

4 Stage III — Discrepancy Protocol 13
- 4.1 Trigger Condition 13
- 4.2 Case A — `LATENT_INCONSISTENCY` 14
- 4.3 Case B — `CRITIC_QUALITY_WARN` 14
- 4.4 No-Trigger Pass-Through 14

5 Stage IV — Composite Diagnostic Scoring 14
- 5.1 Threshold Calibration 14
- 5.2 The Ψ_{CRE} Score 15
- 5.3 Uncertainty Quantification 15
- 5.4 LLM-β: Semantic Analysis and Failure Hypothesis 16

6 Stage V — Repair Generation 16
- 6.1 Repair Space Formalization 16
- 6.2 LLM-δ: Repair Generation 16
- 6.3 Minimality Enforcement 17

7 Stage VI — Acceptance and Validation 17
- 7.1 Multi-Criteria Acceptance Protocol 17
- 7.2 LLM-ε: Semantic Consistency Verification 17
- 7.3 Accept / Reject Decision and Loop Control 17

8 Theoretical Analysis 18
- 8.1 Detection Completeness 18
- 8.2 Algorithm-Agnosticity 19
- 8.3 Repair Loop Convergence 19
- 8.4 Computational Complexity 20

II Agent Developer Handbook 20
- 0.0.1 20

1 Architecture and Global Definitions 21
- 1.1 Module Registry and Pipeline Topology 21
- 1.2 Core Data Schemas 21

1.3 Global Configuration Constants . . . 23
2 Stage I — Module M1: Specification Parsing . . . 24
2.1 Module Overview . . . 24
2.1.1 M1.parse_nl_input . . . 25
2.1.2 M1.detect_and_escalate_ambiguities . . . 26
2.1.3 M1.run_symbolic_precheck . . . 26
3 Stage II — Module M2: Primary Estimators . . . 27
3.1 Module Overview . . . 27
3.2 C–R Dimension . . . 27
3.2.1 M2.compute_phi_cr1 . . . 27
3.2.2 M2.compute_phi_cr2 . . . 28
3.3 E–C Dimension . . . 29
3.3.1 M2.compute_phi_ec_j . . . 29
3.3.2 M2.compute_phi_ec_bar . . . 30
3.3.3 M2.compute_phi_ec2 . . . 30
3.3.4 M2.compute_phi_ec3 . . . 31
3.4 E–R Dimension . . . 31
3.4.1 M2.compute_phi_er1 . . . 31
3.4.2 M2.compute_phi_er2 . . . 32
3.4.3 M2.compute_phi_er3 . . . 32
4 Stage II — Module M3: Enhanced Estimators . . . 33
4.1 Module Overview . . . 33
4.1.1 M3.compute_kappa_cr . . . 33
4.1.2 M3.compute_gamma_ec . . . 34
4.1.3 M3.compute_delta_er . . . 34
5 Stage III — Discrepancy Protocol . . . 35
5.1 Module Overview . . . 35
5.1.1 DP.compute_discrepancy . . . 35
5.1.2 DP.case_a_handler . . . 36
5.1.3 DP.case_b_handler . . . 36
5.1.4 DP.run_protocol . . . 37
6 Stage IV — Module M5: Composite Scoring . . . 38
6.1 Module Overview . . . 38
6.1.1 M5.monotone_transform . . . 38
6.1.2 M5.compute_psi_cre . . . 38
6.1.3 M5.compute_bootstrap_ci . . . 39
6.1.4 M5.calibrate_thresholds . . . 40
7 Stage IV — Module M4: Semantic Analysis . . . 41
7.1 Module Overview . . . 41
7.1.1 M4.generate_failure_hypothesis . . . 41
8 Stage V — Module M7: Repair Generation . . . 42
8.1 Module Overview . . . 42
8.1.1 M7.compute_spec_edit_distance . . . 42
8.1.2 M7.generate_repair_proposals . . . 42
8.1.3 M7.filter_by_minimality . . . 43
8.1.4 M7.rank_proposals . . . 44
9 Stage VI — Module M8: Acceptance and Validation . . . 45
9.1 Module Overview . . . 45
9.1.1 M8.evaluate_acceptance_criteria . . . 45
9.1.2 M8.verify_semantic_consistency . . . 46
9.1.3 M8.run_acceptance_loop . . . 46
10 Pipeline Orchestrator . . . 48
10.1 Module Overview . . . 48
10.1.1 PO.run_cre_pipeline . . . 48

10.1.2 PO.audit_trail_schema 49

11 Error Code Registry 49

12 Integration Test Suite 52

12.1 Module Unit Tests 52

12.1.1 52

12.1.2 Numerical Regression Suite 53

12.1.3 End-to-End Pipeline Tests 53

12.1.4 Termination Rate Benchmark 54

Part I

NeurIPS-Standard Method Section

Scope Boundary: LLM Error Modelling. The theoretical guarantees developed in Part I (Theorems 8.3–8.7) treat all estimators as exact and model LLM-backed modules (M1, M4, M6, M7, M8) as functionally correct semantic translators. LLM error propagation—including specification misparse in M1, confabulatory diagnostic hypotheses in M4/M6, and semantically invalid repair proposals in M7—is **not** formally modelled in the theoretical framework. These failure modes are addressed operationally via retry-with-fallback mechanisms described in Section **??**, and are evaluated empirically in Section **??**. This constitutes a deliberate and acknowledged scope boundary: the formal guarantees in Part I apply to the idealized pipeline, and the practical robustness of LLM-backed components is an empirical, not theoretical, property of the system.

1 Problem Formulation and Notation

1.1 Specification Triple and Formal Representation

We model the pre-training RL design artifact as a *specification triple* that jointly encodes the reward signal, the constraint set, and the environment description. Inconsistencies among these three components are the root cause of reward hacking, unsafe deployment, and distribution-shift failure—yet they are invisible to any post-training diagnostic that inspects only the learned policy. The CRE framework operates at the specification level, detecting and repairing such inconsistencies before or during training.

Definition 1.1 (Specification Triple)**.** *A* specification triple *is the 5-tuple*

$$\mathcal{S} \;=\; \big(\mathcal{E}_{\text{tr}},\; \mathcal{E}_{\text{dep}},\; R,\; \mathcal{C},\; \Pi\big), \tag{1}$$

where $\mathcal{E}_{\text{tr}}$ is the training environment distribution over MDPs $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P_{\text{tr}}, R, \gamma)$; $\mathcal{E}_{\text{dep}} = \{e_0, e_1, \ldots, e_n\}$ is the finite deployment environment set, with e_0 the nominal environment and $e_1, \ldots, e_n$ shifted variants; $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function; $\mathcal{C} = \{(K_j, \sigma_j, h_j)\}_{j=1}^{m}$ is the constraint set (see Definition 1.2); and Π is the policy class.

Definition 1.2 (Constraint Representation)**.** *Each constraint $(K_j, \sigma_j, h_j) \in \mathcal{C}$ comprises:*

- *$K_j \subseteq \mathcal{S} \times \mathcal{A}$: the* indicator set *of violating state-action pairs, encoded as a first-order predicate $\mathcal{O}_j : \mathcal{S} \times \mathcal{A} \to \{0, 1\}$;*
- *$\sigma_j \in \{$**hard**, **soft**, **info**$\}$: the* severity level*; and*
- *$h_j \in \{$**instantaneous**, **episodic**, **cumulative**$\}$: the* temporal scope*.*

A policy π violates *C_j under severity $\sigma_j =$ **hard** if $\mathbb{P}_\pi[\exists\, t : (s_t, a_t) \in K_j$ within scope $h_j] > 0$, and under $\sigma_j =$ **soft** if the expected violation count exceeds tolerance $\zeta_j > 0$.*

The specification $\mathcal{S}$ is constructed by parsing a natural-language document `NLInput` $= (r_{\text{desc}}, c_{\text{desc}}, e_{\text{desc}})$ via the parsing module LLM-α (Stage I, Section 2). The parsed $\mathcal{S}$ is *immutable* throughout the diagnostic pipeline; only after a validated repair cycle (Stages V–VI) is it replaced by $\mathcal{S}'$.

1.2 Three-Class Inconsistency Taxonomy

Definition 1.3 (Class I: Constraint–Reward Inconsistency, C–R)**.** *$\mathcal{S}$ exhibits a* Class I inconsistency *if*

$$\exists\, \pi \in \Pi : \quad J(\pi) > J(\pi^*) - \epsilon_J \;\; \textit{and} \;\; \mathbb{P}_\pi\big[\exists\, t : (s_t, a_t) \in K_j\big] > 0 \;\; \textit{for some } C_j \textit{ with } \sigma_j = \texttt{hard}, \tag{2}$$

where $J(\pi) = \mathbb{E}_\pi[\sum_t \gamma^t R(s_t, a_t)]$ and $\epsilon_J \geq 0$ is a near-optimality tolerance. This formalizes two failure modes: (i) boundary-seeking*: the near-optimal policy operates in states arbitrarily close to ∂K_j; and (ii)* gradient opposition*: the reward and safety gradients oppose each other, i.e.* $\cos(\nabla_\theta J_R, \nabla_\theta J_C) < -\delta_{\text{opp}}$ *for some $\delta_{\text{opp}} > 0$.*

Remark 1.4 (Gap Between Definition and Estimator)**.** *Definition 1.3 posits an* existential *condition: a near-optimal policy that simultaneously achieves high reward and violates a hard constraint C_j. The canonical reporter $\phi^{(2)}_{CR}$ (boundary-seeking score, Section 3.3) serves as an operational surrogate for this condition under geometric boundary-type safety constraints. It is a* ***sufficient but not necessary*** *witness: a high $\phi^{(2)}_{CR}$ provides positive evidence consistent with Definition 1.3, but low boundary-seeking behavior does not rule out other manifestations of C–R inconsistency.*

Specifically, the following configurations satisfy Definition 1.3 but are ***not*** *reliably detected by $\phi^{(2)}_{CR}$ alone:*

- *Reward functions that incentivize* sporadic *hard violations without sustained boundary proximity (the policy occasionally violates but does not persistently approach the boundary);*
- *Conflicts between the reward and* cumulative *soft constraints, where the tension does not manifest as geometric proximity to any single constraint boundary;*
- *Constraints defined over the* action space *rather than a geometric state boundary, for which boundary proximity is not a meaningful signal.*

The near-optimal tolerance ϵ_J in Definition 1.3 additionally introduces threshold sensitivity: shrinking ϵ_J toward zero may reclassify genuinely inconsistent specifications as consistent by excluding policies that achieve slightly sub-optimal reward while violating constraints. This parameter should be calibrated with awareness of the flatness of the reward landscape near the optimum. For domains where the above failure modes are of concern, domain-specific reporters should be registered via the extension interface described in Section **??**.

Definition 1.5 (Class II: Environment–Constraint Inconsistency, E–C)**.** *$\mathcal{S}$ exhibits a* Class II inconsistency *if*

$$\exists\, j \in \{1, \ldots, m\} : \quad \mathrm{Cov}(K_j, \mathcal{E}_{\text{tr}}) \;:=\; \frac{\big|\{s \in K_j \mid \exists\, \tau \sim \pi_{\text{ref}} \textit{ in } \mathcal{E}_{\text{tr}} : s \in \tau\}\big|}{|K_j|} \;<\; \delta_j, \tag{3}$$

where $\delta_j > 0$ is the minimum training-coverage threshold for constraint C_j. Informally, the training environment lacks trajectories that visit the critical constraint region K_j with sufficient frequency to provide a learning signal for constraint satisfaction.

Remark 1.6 (Measure-Theoretic Grounding of Coverage)**.** *The coverage ratio* $\mathrm{Cov}(K_j, \mathcal{E}_{tr})$ *in Definition 1.5 requires a well-defined measure on the critical region K_j:*

- ***Finite/discrete state spaces.*** *$|K_j|$ is taken as the set cardinality, and coverage reduces to a simple frequency ratio. The definition is exact.*

- ***Continuous state spaces.*** *$|K_j|$ must be interpreted as a Lebesgue measure $\mu(K_j)$ with respect to a reference density on the state space. In this case, the coverage estimate depends on both the choice of sampling density and the membership radius r_j used in the approximation.*

*In the operational implementation (Section 3.4), K_j membership is approximated via KD-tree nearest-neighbour lookup with radius r_j, using rejection sampling against a reference policy rollout. This introduces estimator bias that increases with r_j and depends on the local geometry of K_j: when K_j is a thin or high-dimensional manifold, radius-based membership can overestimate coverage. The parameter r_j is exposed as a configuration hyperparameter (`critical_region_radius`, Section **??**); practitioners in high-dimensional state spaces should conduct sensitivity analysis over r_j. Formal bias-variance analysis of the MC estimator under varying r_j is left for future work.*

Definition 1.7 (Class III: Environment–Reward Inconsistency, E–R)**.** *$\mathcal{S}$ exhibits a* Class III inconsistency *if*

$$\phi_{ER}^{(3)}(\mathcal{S}) \;:=\; \frac{1-\rho_{RU}(\pi^*;\mathcal{E}_{\text{dep}})}{2} \;>\; \theta_{\text{ER}}, \tag{4}$$

where

$$\rho_{RU}(\pi^*;\mathcal{E}_{\text{dep}}) \;:=\; \text{Corr}_{e\in\mathcal{E}_{\text{dep}}}\Big(R_{\text{norm}}(\pi^*;\, e),\; U(\pi^*;\, e)\Big) \in [-1,1], \tag{5}$$

$R_{\text{norm}}(\pi; e) \in [0,1]$ is the normalized cumulative reward of π in environment e, $U(\pi; e) \in [0,1]$ is an independent semantic utility score, and $\theta_{\text{ER}} \in (0,1)$ is the decoupling threshold. This formalizes cross-environment reward–utility decoupling*: the reward ranking induced across deployment environments fails to track the corresponding utility ranking.*

1.3 Formal Problem Statement: Detection, Localization, and Repair

The *CRE diagnostic and repair problem* comprises three coupled sub-problems.

Postulate 1 (P1): Graded Inconsistency Detection. Let $\Psi_{CRE}(S) \in [0,1]$ be the CRE score of a source set S, and let

$$\tau_{alarm} \;\leq\; \tau_{detect} \tag{6}$$

be two scalar thresholds defined in §5.1. We distinguish two levels of inconsistency:

(i) **Detectable inconsistency.** S is said to contain a *detectable inconsistency* if and only if

$$\Psi_{CRE}(S) \;<\; \tau_{detect}. \tag{7}$$

(ii) **Actionable (alarm-level) inconsistency.** S is said to contain an *actionable inconsistency*—triggering a system alarm—if and only if

$$\Psi_{CRE}(S) \;<\; \tau_{alarm}. \tag{8}$$

The original single-threshold formulation "$\Psi_{CRE}(S) < \tau_{alarm}$ iff at least one inconsistency holds" is a special case obtained by setting $\tau_{detect} = \tau_{alarm}$; however, this collapses the conceptually distinct notions of *existence* and *severity* of inconsistency, and is therefore replaced by the graded formulation above.

Remark 1.8. *Every actionable inconsistency is also detectable (since $\tau_{alarm} \leq \tau_{detect}$ implies $\Psi_{CRE}(S) < \tau_{alarm} \Rightarrow \Psi_{CRE}(S) < \tau_{detect}$), but the converse does not hold in general. A source set may exhibit a detectable inconsistency without crossing the alarm threshold.*

P2: Localization. Identify the active dimension(s) $d^* \subseteq \{\mathrm{CR}, \mathrm{EC}, \mathrm{ER}\}$, and produce witness evidence (trajectory segments, state-index sets, or activation count vectors) that substantiate each diagnosis.

P3: Repair with Minimality. Construct a modified specification $\mathcal{S}'$ satisfying: (i) $\Psi_{\mathrm{CRE}}(\mathcal{S}') > \Psi_{\mathrm{CRE}}(\mathcal{S})$ (diagnostic improvement); (ii) the semantic task intent is preserved; and (iii) $d_{\mathrm{spec}}(\mathcal{S}, \mathcal{S}') \leq \eta_{\mathrm{edit}}$ (minimal edit), where d_{spec} is the specification edit distance (Definition 6.2).

2 Stage I — Specification Parsing and Representation

2.1 LLM-α: Parse and Ambiguity Detection

Module M1 transforms the unstructured input `NLInput` into the typed structured representation $\mathcal{S}$, and simultaneously detects semantic ambiguities that would propagate as noise into downstream estimators.

Definition 2.1 (Ambiguity Flag)**.** *An* ambiguity flag $\mathcal{F} = (t, \ell, f_{\mathrm{imp}})$ *is a triple of flag type* $t \in \mathcal{T}_{\mathrm{flag}}$*, field location* ℓ *(JSON path within* $\mathcal{S}$*), and downstream impact score* $f_{\mathrm{imp}} \in [0, 1]$ *estimating the maximal fractional reduction in* Ψ_{CRE} *reliability if* ℓ *remains unresolved. The flag-type vocabulary is:*

$\mathcal{T}_{\mathrm{flag}} = \{$*UNDEFINED_THRESHOLD*, *MISSING_ENV_SCOPE*, *REWARD_UNDERSPECIFIED*, *CONSTRAINT_OVERLAP*, *TEMP*

The impact score f_{imp} is obtained from a lightweight sensitivity model: a gradient-boosted regressor trained on 1,200 annotated specification pairs, mapping flag type and field path to observed $|\Delta\Psi_{\mathrm{CRE}}|$ degradation. Flags with $f_{\mathrm{imp}} > \eta_{\mathrm{flag}}$ (default 0.15) are escalated before estimation proceeds.

The parsing prompt $\mathcal{P}_\alpha$ (full template in Appendix B.1) instructs LLM-α to: (1) extract all reward terms into a typed weighted graph; (2) enumerate constraint indicator predicates with explicit state-action subspaces; (3) characterize the environment via scene families and parametric shift operators; (4) assign ambiguity flags with computed impact scores. Fields that cannot be resolved from the natural-language input are populated with typed `null` values.

2.2 Structured Specification Builder

Reward Representation. The reward function is encoded as a weighted DAG $G_R = (V_R, E_R)$: each node $v \in V_R$ is a term $r_k : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ carrying weight w_k, clip bounds $[c_k^-, c_k^+]$, and a shaping flag. The composite reward is

$$R(s, a) = \sum_k w_k \cdot \mathrm{clip}\big(r_k(s, a),\, c_k^-, c_k^+\big). \tag{9}$$

Edges in E_R encode potential shaping dependencies (e.g., r_k is a potential-based shaping term derived from $r_{k'}$).

Constraint Representation. Each K_j is encoded as a first-order predicate over $(\mathcal{S}, \mathcal{A})$, callable at runtime via $\mathcal{O}_j : \mathcal{S} \times \mathcal{A} \to \{0, 1\}$. Severity and temporal scope are stored as typed enum fields. A penalty weight λ_j is optionally provided for soft constraints.

Environment Representation. The environment is encoded as a family of scene descriptors $\mathcal{F} = \{f_1, \ldots, f_p\}$, a set of shift operators $\{\tau_i\}_{i=1}^q$ (perturbations mapping $\mathcal{E}_{\mathrm{tr}} \to e_i \in \mathcal{E}_{\mathrm{dep}}$), and a list of generator seeds for reproducible instantiation. Scene families cover nominal and deployment conditions jointly.

2.3 Internal Consistency Pre-Check

Before proceeding to estimation, a symbolic pre-check validates: (1) *Type compatibility*: all reward terms and constraint predicates operate on compatible subspaces of $\mathcal{S} \times \mathcal{A}$; (2) *Domain boundary*: the support of R intersects the interior of $\mathcal{S} \setminus \bigcup_j K_j$ (otherwise a trivial repair exists); (3) *Coverage pre-bound*: $|\mathcal{F}|$ provides at least one scene family per hard constraint C_j. Failures that cannot be auto-resolved are escalated to `AMBIGUITY_UNRESOLVABLE`.

3 Stage II — Two-Axis Estimation Framework

3.1 Axis Definitions and Theoretical Motivation

The estimation framework organizes all inconsistency metrics along two orthogonal conceptual axes governing their properties and roles.

Axis 1 — Abstraction Level. *Behavioral estimators* (Level B) measure *what happens* during trajectory rollouts: observable statistics such as reward correlations, state visitation distributions, and performance gaps across environments. *Mechanistic estimators* (Level M) measure *why* inconsistencies arise: properties of the policy optimization landscape, such as gradient directions and critic-derived value signals, that are causally upstream of behavioral symptoms. Level B estimators require only trajectory data; Level M estimators additionally require access to the policy gradient or a trained critic.

Axis 2 — Algorithm Dependence. *Optimizer-agnostic estimators* (Class Ag) make no assumption about the specific RL algorithm, policy parameterization, or critic architecture. *Critic-dependent estimators* (Class Cd) require a trained value function V_ϕ or Q-function Q_ϕ and thus implicitly depend on the training algorithm.

The four quadrants (B,Ag), (B,Cd), (M,Ag), (M,Cd) span the estimator design space. Primary estimators occupy (B,Ag) and (B,Cd)—maximally accessible. Enhanced estimators occupy (M,Ag) and (M,Cd)—providing mechanistic depth for the Discrepancy Protocol (Stage III).

3.2 Canonical Reporter Selection

The three reporters designated below are the **default canonical reporters** for their respective inconsistency classes under the following baseline domain conditions:

(i) safety constraints are expressed as geometric state-boundary conditions amenable to boundary-proximity measurement;

(ii) the state space admits a well-defined notion of critical-region membership;

(iii) a faithful utility oracle satisfying Assumptions 3.2–3.4 (Section 3.6) is available.

Under these conditions and the ideal-oracle assumptions of Theorem 8.3, the three reporters jointly satisfy coverage, parsimony, algorithm-independence, and internal uniqueness *within the framework's operational definitions.*

C–R reporter: $\phi^{(2)}_{CR}$ (boundary-seeking score). This is the default canonical reporter for C–R inconsistency under *geometric boundary-type* safety constraints. This designation does not extend to all forms of reward–constraint mismatch; in particular, it does not cover reward-driven sporadic violations without boundary proximity, cumulative soft-constraint conflicts, or constraints defined over action spaces rather than state geometry. For such settings,

domain-specific reporters should be registered via the extension interface (Section **??**); the designation should then be understood as "default canonical reporter under boundary-type safety constraints."

E–C reporter: $\bar{\phi}^{(1)}_{EC}$ (mean critical-state coverage). This is the default canonical reporter for E–C inconsistency. As discussed in Section 3.4.1, mean aggregation can mask worst-case undercoverage on individual hard constraints. For safety-critical deployments, the minimum aggregation mode (`ec_aggregation_mode: "min"`) is recommended, and per-constraint coverage values $\phi^{(1,j)}_{EC}$ should be inspected individually.

E–R reporter: $\phi^{(3)}_{ER}$ (reward–utility decoupling score). This is the default canonical reporter for E–R inconsistency, subject to the oracle assumptions in Section 3.6. It is the reporter most dependent on external system commitments; its designation as "canonical" is contingent on the availability of a faithful independent utility oracle.

Canonical Reporter Assignments. $\varphi^{(2)}_{\mathrm{CR}}$ (boundary-seeking score) is canonical for C–R. $\bar{\varphi}^{(1)}_{\mathrm{EC}}$ (mean critical-state coverage) is canonical for E–C. $\varphi^{(3)}_{\mathrm{ER}}$ (reward–utility decoupling score) is canonical for E–R. Only these three estimators enter the Ψ_{CRE} computation. All other estimators are *supplementary* (provide diagnostic context) or *enhanced* (feed the Discrepancy Protocol only).

The algorithm-agnosticity of Ψ_{CRE} follows directly: all three canonical reporters lie in Class Ag; their values depend on π^*, $\mathcal{E}_{\mathrm{tr}}$, and $\mathcal{S}$, but not on any specific optimizer or critic. This is formalized in Theorem 8.5.

3.3 Primary Estimators — Constraint–Reward Dimension

Supplementary: $\varphi^{(1)}_{\mathrm{CR}}$ — Reward-Risk Correlation.

$$\varphi^{(1)}_{\mathrm{CR}} = \frac{\mathrm{Cov}_{\tau\sim\pi^*}\Big(\sum_t \gamma^t R(s_t,a_t),\ \mathbf{1}\big[\exists\, t : (s_t,a_t) \in \bigcup_j K_j\big]\Big)}{\sigma_R \cdot \sigma_C}, \tag{10}$$

where π^* is the reward-optimal policy under $\mathcal{E}_{\mathrm{tr}}$. A positive value indicates that high-reward trajectories are disproportionately associated with constraint violations.

Canonical: $\varphi^{(2)}_{\mathrm{CR}}$ — Boundary-Seeking Score.

$$\varphi^{(2)}_{\mathrm{CR}} = \max\left(0,\ 1 - \frac{\mathbb{E}_{\pi^*,\,\tau\sim\mathcal{E}_{\mathrm{tr}}}\Big[\min_{j:\,\sigma_j=\texttt{hard}} d_{\mathcal{S}}(s_t,\partial K_j)\Big]}{\mathbb{E}_{\pi_{\mathrm{unif}},\,\tau\sim\mathcal{E}_{\mathrm{tr}}}\Big[\min_{j:\,\sigma_j=\texttt{hard}} d_{\mathcal{S}}(s_t,\partial K_j)\Big]}\right), \tag{11}$$

where $d_{\mathcal{S}}$ is the state-space Euclidean distance, $\partial K_j = \{s : \mathcal{O}_j(s,a) = 1 \text{ for some } a\}$ is the constraint boundary approximated by nearest-neighbor lookup, and π_{unif} is the uniform random policy baseline. The $\max(0,\cdot)$ truncation ensures $\varphi^{(2)}_{\mathrm{CR}} \in [0,1]$: when π^* maintains strictly greater average boundary clearance than π_{unif}, the raw ratio exceeds one and the score is floored at zero, indicating no boundary-seeking relative to the baseline. $\varphi^{(2)}_{\mathrm{CR}} = 0$ indicates that π^* is at least as safe as π_{unif} (no boundary-seeking relative to the random-policy baseline); $\varphi^{(2)}_{\mathrm{CR}} \to 1$ indicates that π^* has near-zero expected boundary clearance (maximally boundary-seeking).

Proposition 3.1. *$\varphi^{(2)}_{\mathrm{CR}}(\mathcal{S}) = 0$ if and only if π^* maintains at least as great an expected distance to all hard constraint boundaries as π_{unif} (boundary-seeking is no worse than the random baseline). $\varphi^{(2)}_{\mathrm{CR}}(\mathcal{S}) = 1$ if and only if π^* achieves zero expected distance to the nearest hard constraint boundary under $\mathcal{E}_{\mathrm{tr}}$ (maximally boundary-seeking).*

3.4 Primary Estimators — Environment–Constraint Dimension

Canonical: $\bar{\varphi}^{(1)}_{\mathrm{EC}}$ — Mean Critical-State Coverage. For each hard constraint C_j, define the per-constraint coverage:

$$\varphi^{(1,j)}_{\mathrm{EC}} = \frac{\left|\{s \in K_j \mid \exists \tau \sim \pi_{\mathrm{ref}},\, \tau \in \mathcal{D}_{\mathcal{E}_{\mathrm{tr}}} : s \in \tau\}\right|}{|K_j|}, \tag{12}$$

where $\mathcal{D}_{\mathcal{E}_{\mathrm{tr}}}$ is the trajectory dataset collected under $\mathcal{E}_{\mathrm{tr}}$ using a lightly-trained reference policy π_{ref}, and $|K_j|$ is estimated by Monte Carlo sampling of the constraint predicate. The canonical aggregate is:

$$\bar{\varphi}^{(1)}_{\mathrm{EC}} = \frac{1}{m_{\mathrm{hard}}} \sum_{j:\, \sigma_j = \texttt{hard}} \varphi^{(1,j)}_{\mathrm{EC}}, \quad m_{\mathrm{hard}} = |\{j : \sigma_j = \texttt{hard}\}|. \tag{13}$$

$\bar{\varphi}^{(1)}_{\mathrm{EC}} \in [0,1]$ measures coverage: high values indicate adequate exposure to hard constraint regions; low values indicate Class II undercoverage (Definition 1.5). The corresponding *undercoverage* term $1 - \bar{\varphi}^{(1)}_{\mathrm{EC}} \in [0,1]$ enters Ψ_{CRE} as the E–C inconsistency signal (see Definition 5.1).

Supplementary: $\varphi^{(2)}_{\mathrm{EC}}$ — Constraint Activation Frequency.

$$\varphi^{(2)}_{\mathrm{EC}} = \mathbb{E}_{\tau \sim \pi^*, \mathcal{E}_{\mathrm{tr}}}\left[\frac{1}{T}\sum_{t=0}^{T-1} \mathbf{1}\big[(s_t, a_t) \in \textstyle\bigcup_j K_j\big]\right]. \tag{14}$$

Supplementary: $\varphi^{(3)}_{\mathrm{EC}}$ — Constraint Diversity.

$$\varphi^{(3)}_{\mathrm{EC}} = H\left(\left\{\mathbb{E}_\tau\big[\mathbf{1}[(s_t, a_t) \in K_j]\big]\right\}_{j=1}^{m}\right) \Big/ \log m, \tag{15}$$

where $H(\cdot)$ is the Shannon entropy, normalized to $[0,1]$ by $\log m$. Low diversity indicates that only a subset of constraints are ever activated in $\mathcal{E}_{\mathrm{tr}}$.

3.4.1 E–C Aggregation Mode

By default, $\bar{\phi}^{(1)}_{EC}$ is computed as the unweighted mean of per-constraint coverage scores:

$$\bar{\phi}^{(1)}_{EC} = \frac{1}{|\mathcal{J}|}\sum_{j \in \mathcal{J}} \phi^{(1,j)}_{EC}.$$

This *mean* mode provides a balanced summary appropriate for general-purpose evaluation. However, mean aggregation can suppress worst-case undercoverage: a hard constraint C_j with zero or near-zero coverage may be offset by high coverage on other constraints, leaving the most critical failure mode invisible in the composite score.

For safety-critical deployments, the framework supports a *minimum* aggregation mode over hard constraints:

$$\bar{\phi}^{(1,\min)}_{EC} = \min_{j:\, \sigma_j = \texttt{hard}} \phi^{(1,j)}_{EC}.$$

This mode guarantees that insufficient coverage on any single hard constraint is directly reflected in $\bar{\phi}_{EC}^{(1)}$, preventing dilution through averaging. The aggregation mode is controlled by the configuration field `ec_aggregation_mode` (values: `"mean"` | `"min"`; default: `"mean"`). Domain presets with `safety_level: critical` automatically set `ec_aggregation_mode: "min"`.

Regardless of the chosen aggregation mode, it is strongly recommended that practitioners in safety-critical settings inspect per-constraint coverage values $\phi_{EC}^{(1,j)}$ individually as part of the diagnostic report, since even the minimum-mode aggregate may not convey the full distribution of coverage deficiencies across constraint dimensions.

3.5 Primary Estimators — Environment–Reward Dimension

Supplementary: $\varphi_{\text{ER}}^{(1)}$ — Deployment Utility Gap.

$$\varphi_{\text{ER}}^{(1)} \;=\; U_{\text{nom}}(\pi^*) - \frac{1}{n}\sum_{i=1}^{n} U(\pi^*;\, e_i), \tag{16}$$

where $U_{\text{nom}}(\pi^*) = U(\pi^*;\, e_0)$ is the nominal utility and $U(\pi^*;\, e_i)$ is utility in deployment environment e_i.

Supplementary: $\varphi_{\text{ER}}^{(2)}$ — Deployment Reward Gap.

$$\varphi_{\text{ER}}^{(2)} \;=\; R_{\text{norm}}(\pi^*;\, e_0) - \frac{1}{n}\sum_{i=1}^{n} R_{\text{norm}}(\pi^*;\, e_i), \tag{17}$$

where $R_{\text{norm}}(\pi;\, e) \in [0, 1]$ is the normalized cumulative reward of π in e.

Canonical: $\varphi_{\text{ER}}^{(3)}$ — Reward–Utility Decoupling Score.

$$\varphi_{\text{ER}}^{(3)} \;=\; \frac{1}{2}\left(1 - \frac{\sum_{i=0}^{n}\big(R_{\text{norm}}(\pi^*;\, e_i) - \bar{R}\big)\big(U(\pi^*;\, e_i) - \bar{U}\big)}{\sqrt{\sum_i\big(R_{\text{norm}}(\pi^*;\, e_i) - \bar{R}\big)^2 \cdot \sum_i\big(U(\pi^*;\, e_i) - \bar{U}\big)^2}}\right), \tag{18}$$

where $\bar{R}, \bar{U}$ are cross-environment sample means. By construction, $\varphi_{\text{ER}}^{(3)} \in [0, 1]$: $\varphi_{\text{ER}}^{(3)} = 0$ indicates perfect reward–utility alignment, $\varphi_{\text{ER}}^{(3)} = 0.5$ indicates zero correlation, and $\varphi_{\text{ER}}^{(3)} = 1$ indicates maximal anti-correlation across deployment environments. This normalized form enters Ψ_{CRE} directly without any additional clipping step.

3.6 Utility Oracle Assumptions

The reporter $\phi_{ER}^{(3)}$ measures the divergence between reward-ranked and utility-ranked trajectory distributions, relying on a behavioural utility oracle $U(\pi;\, e)$ that scores policy π's task performance on episode e independently of the training reward signal. The theoretical treatment in Section 8 assumes this oracle is *faithful*—accurately reflecting the ground-truth task objective. This faithfulness is not a mathematical consequence of the framework; it is a system design commitment. The following assumptions make the required conditions explicit.

Assumption 3.2 (Independence of Construction)**.** *The oracle U must be specified prior to and independently of the reward function R. Acceptable constructions include:*

(i) programmatic verification such as task-completion flags or rule-based safety monitors;

(ii) structured human evaluation protocols using annotation guidelines that do not reference the reward specification;

(iii) a separate LLM evaluator conditioned on a task description that is disjoint from the reward design process.

Any construction that derives U from R by monotone transformation, shared labelling, or common annotator violates the independence required for a valid E–R measurement (Definition 1.7) and renders $\phi_{ER}^{(3)}$ uninformative.

Assumption 3.3 (Statistical Independence from R)**.** *The E–R measurement is meaningful only if U and R are statistically independent in their construction. Practitioners must verify that no shared model, dataset, annotation source, or automated labeller exists between the reward specification pipeline and the utility oracle pipeline. Where complete independence is infeasible, the degree of correlation should be documented and treated as a limitation on the interpretability of $\phi_{ER}^{(3)}$.*

Assumption 3.4 (Task Objective Coverage)**.** *A scalar task-completion rate is sufficient for single-objective tasks. For multi-objective tasks, U should be extended to a vector-valued oracle $\mathbf{U} = (U_1, \ldots, U_K)$, and $\phi_{ER}^{(3)}$ should be computed as:*

$$\phi_{ER}^{(3)} = \max_{k \in [K]} \phi_{ER,k}^{(3)}$$

to ensure that misalignment on any task dimension is surfaced. The framework's `utility_oracle_config` field (Section 1.2) supports multi-dimensional oracle specification; see Appendix ?? for configuration examples.

When Assumptions 3.2–3.4 cannot be fully satisfied, $\phi_{ER}^{(3)}$ should be treated as a heuristic diagnostic indicator rather than a formal E–R inconsistency measure, and findings should be validated through independent behavioural audits.

3.7 Non-Degeneracy Conventions for Ratio and Correlation Reporters

The reporters $\phi_{CR}^{(1)}$, $\phi_{CR}^{(2)}$, and $\phi_{ER}^{(3)}$ contain denominators that may vanish on degenerate datasets. The framework adopts the following conventions.

- For $\phi_{CR}^{(1)}$ in Eq. (10), if $\sigma_R \sigma_C = 0$, the reporter is set to 0 and a `DEGENERATE_CR1_VAR` flag is attached. Since $\phi_{CR}^{(1)}$ is supplementary, this convention affects diagnostics only and does not enter Ψ_{CRE}.
- For $\phi_{CR}^{(2)}$ in Eq. (11), the denominator requires strictly positive expected boundary clearance under π_{unif}. If

$$\mathbb{E}_{\pi_{\text{unif}}, \tau \sim \mathcal{E}_{\text{tr}}} \Big[\min_{j:\, \sigma_j = \texttt{hard}} d_{\mathcal{S}}(s_t, \partial K_j) \Big] = 0,$$

  the source set is deemed boundary-baseline degenerate and M2 raises `BOUNDARY_BASELINE_DEGENERATE`.
- For $\phi_{ER}^{(3)}$ in Eq. (18), if either cross-environment variance term in the denominator is zero, the utility oracle/environment pair is deemed correlation-degenerate and M2 raises `ER_CORRELATION_DEGENERATE`. In that case the source set must be repaired at the oracle/configuration level before a valid E–R score can be issued.

These conventions make the scoring pipeline numerically well-defined without changing the role of canonical reporters in Ψ_{CRE}.

3.8 Enhanced Estimators — Mechanistic Layer

$\tilde{\kappa}_{\mathrm{CR}}$ — **Gradient Conflict Score (B/M, Ag).**

$$\tilde{\kappa}_{\mathrm{CR}} \;=\; \mathbb{E}_{\pi^*,\,\tau\sim\mathcal{E}_{\mathrm{tr}}}\Big[-\cos\Big(\nabla_\theta J_R(\pi_\theta),\ \nabla_\theta J_C(\pi_\theta)\Big)\Big]\,, \tag{19}$$

where $J_C = -\sum_j \mathbb{E}_\pi[\mathbf{1}[(s_t,a_t)\in K_j]]$. $\tilde{\kappa}_{\mathrm{CR}} > 0$ indicates that improving reward requires moving the policy in a direction that worsens constraint satisfaction—the mechanistic signature of Class I.

$\tilde{\gamma}_{\mathrm{EC}}$ — **Bisimulation Soft Coverage (M, Ag).**

$$\tilde{\gamma}_{\mathrm{EC}} \;=\; 1 - \frac{1}{m_{\mathrm{hard}}} \sum_{j:\,\sigma_j=\texttt{hard}} \mathbb{E}_{s\sim\mathcal{D}_{\mathcal{E}_{\mathrm{dep}}}}\Big[\max_{s'\in K_j} \phi_{\mathrm{sim}}(s,s')\Big]\,, \tag{20}$$

where $\phi_{\mathrm{sim}} : \mathcal{S}\times\mathcal{S}\to[0,1]$ is a bisimulation similarity metric learned contrastively. High $\tilde{\gamma}_{\mathrm{EC}}$ indicates poor soft coverage even accounting for states *similar* to the critical set.

$\tilde{\delta}_{\mathrm{ER}}$ — **Gradient Drift Score (M, Cd).**

$$\tilde{\delta}_{\mathrm{ER}} \;=\; \max_{i\in\{1,\dots,n\}} \frac{\big\|\nabla_\theta R(\tau;\,e_i) - \nabla_\theta R(\tau;\,e_0)\big\|_2}{\big\|\nabla_\theta R(\tau;\,e_0)\big\|_2 + \epsilon_{\mathrm{stab}}}, \tag{21}$$

normalized by the nominal gradient norm for scale invariance. Large $\tilde{\delta}_{\mathrm{ER}}$ indicates that the reward gradient direction shifts substantially across environments—the mechanistic cause of reward–utility decoupling.

Role Constraint — Immutability Principle. Enhanced estimators $\tilde{\kappa}_{\mathrm{CR}}$, $\tilde{\gamma}_{\mathrm{EC}}$, $\tilde{\delta}_{\mathrm{ER}}$ are computed by Module M3 and routed exclusively to the Discrepancy Protocol (Stage III). They *never* enter the Ψ_{CRE} numerical computation. Any code path that writes an enhanced estimator value into a canonical slot raises a `CanonicalViolationError` and halts the pipeline. This is a hard architectural invariant, not a soft recommendation.

4 Stage III — Discrepancy Protocol

4.1 Trigger Condition

The Discrepancy Protocol is a conditional module that activates only when both primary (M2) and enhanced (M3) estimators are available. For each dimension $d\in\{\mathrm{CR},\mathrm{EC},\mathrm{ER}\}$, the discrepancy measure is

$$\Delta_d \;=\; \big|\hat{\varphi}_d - \hat{\kappa}_d\big|, \tag{22}$$

where $\hat{\varphi}_d$ is the canonical primary estimator normalized to $[0,1]$ and $\hat{\kappa}_d$ is the corresponding enhanced estimator (see Table 1). The protocol triggers for dimension d iff $\Delta_d > \eta_{\mathrm{disc}}$ (default $\eta_{\mathrm{disc}} = 0.15$).

Table 1: Primary–enhanced estimator pairs for the Discrepancy Protocol.

Dim.	Canonical $\hat{\varphi}_d$	Enhanced $\hat{\kappa}_d$	Axis	Requires Critic
C–R	$\varphi^{(2)}_{\mathrm{CR}}$	$\tilde{\kappa}_{\mathrm{CR}}$	(B,Ag)+(M,Ag)	No
E–C	$\bar{\varphi}^{(1)}_{\mathrm{EC}}$	$\tilde{\gamma}_{\mathrm{EC}}$	(B,Ag)+(M,Ag)	No
E–R	$\varphi^{(3)}_{\mathrm{ER}}$	$\tilde{\delta}_{\mathrm{ER}}$	(B,Ag)+(M,Cd)	Yes

4.2 Case A — LATENT_INCONSISTENCY

Case A triggers when simultaneously $\hat{\varphi}_d < \tau_{\text{low}}$, $\hat{\kappa}_d > \tau_{\text{high}}$, and $\Delta_d > \eta_{\text{disc}}$. This pattern indicates an incipient inconsistency that is not yet visible in behavioral statistics but is mechanistically present in the gradient structure of the policy.

Invariant 4.1 (Ψ_{CRE} Immutability Under Case A)**.** *When Case A triggers for dimension d, the Ψ_{CRE} point estimate is* not modified. *The canonical primary value $\hat{\varphi}_d$ is held fixed. Instead, only the uncertainty interval for dimension d is expanded:*

$$\sigma_d \leftarrow \sigma_d \cdot (1 + \lambda \cdot \Delta_d), \tag{23}$$

where $\lambda > 0$ is the expansion coefficient (default 0.5*). Additionally,* LLM-γ *generates a temporal hypothesis predicting the training step at which the inconsistency becomes behaviorally manifest (prompt template in Appendix B.3).*

4.3 Case B — CRITIC_QUALITY_WARN

Case B triggers when $\hat{\varphi}_d > \tau_{\text{high}}$, $\hat{\kappa}_d < \tau_{\text{low}}$, and $\Delta_d > \eta_{\text{disc}}$. This pattern indicates that the critic or gradient estimator may be unreliable (e.g., due to insufficient critic training epochs). Under Case B, the canonical primary value *dominates*: $\hat{\varphi}_d$ is held unchanged, a `CRITIC_QUALITY_WARN` flag is appended, and a re-evaluation checkpoint is scheduled at iteration N_{rechk} (default 1).

4.4 No-Trigger Pass-Through

When $\Delta_d \leq \eta_{\text{disc}}$ for all d, the protocol terminates immediately, leaving all estimator records and uncertainty intervals unchanged. A null-discrepancy log entry is written to the audit trail.

5 Stage IV — Composite Diagnostic Scoring

5.1 Threshold Calibration

We formally define and operationalise the two thresholds introduced in Postulate 1 (Section 1.3).

Detection threshold τ_{detect}. The detection threshold marks the boundary below which a drop in $\Psi_{CRE}(S)$ is deemed statistically non-negligible—i.e. unlikely to arise from sampling noise alone. Let $\mu_0 = \mathbb{E}[\Psi_{CRE}(S)]$ and $\sigma_0 = \text{Std}[\Psi_{CRE}(S)]$ be the mean and standard deviation of the score under a null distribution of consistent sources (estimated on a held-out calibration corpus). We set

$$\tau_{detect} = \mu_0 - k_d \sigma_0, \qquad k_d > 0, \tag{24}$$

where k_d is a sensitivity parameter controlling the false-positive rate for detection. A recommended default is $k_d = 1.5$, yielding approximately 7% false-positive rate under a Gaussian null.

Alarm threshold τ_{alarm}. The alarm threshold is calibrated against *operational cost* rather than statistical significance. Let c_{fp} and c_{fn} denote the costs of a false alarm and a missed alarm, respectively. Under a decision-theoretic criterion, the optimal alarm threshold satisfies

$$\tau_{alarm} = \arg\min_{\tau} \Big[c_{fp} \cdot \Pr\big(\Psi_{CRE}(S) < \tau \mid S \text{ consistent}\big) + c_{fn} \cdot \Pr\big(\Psi_{CRE}(S) \geq \tau \mid S \text{ inconsistent}\big) \Big]. \tag{25}$$

In practice, when cost estimates are unavailable, a conservative default of $\tau_{alarm} = \mu_0 - k_a\,\sigma_0$ with $k_a > k_d$ is used, so that the ordering constraint $\tau_{alarm} \leq \tau_{detect}$ (Equation 6) is automatically satisfied.

Relationship between the two thresholds. Table 2 summarises the semantic roles of the two thresholds.

Table 2: Semantic distinction between τ_{detect} and τ_{alarm}.

Threshold	Condition triggered	Calibration basis	Typical value
τ_{detect}	Detectable inconsistency exists	Statistical (null distribution)	$\mu_0 - 1.5\,\sigma_0$
τ_{alarm}	Inconsistency requires action	Operational cost (c_{fp}, c_{fn})	$\mu_0 - 3.0\,\sigma_0$

5.2 The Ψ_{CRE} Score

Definition 5.1 (Ψ_{CRE} Score)**.** *Let $f : [0,1] \to [0,1]$ be the monotone transform defined in (27). The* CRE composite diagnostic score *is*

$$\boxed{\Psi_{\text{CRE}}(\mathcal{S}) \;=\; 1 - \Big[w_{\text{CR}}\, f\big(\varphi^{(2)}_{\text{CR}}(\mathcal{S})\big) + w_{\text{EC}}\, f\big(1 - \bar{\varphi}^{(1)}_{\text{EC}}(\mathcal{S})\big) + w_{\text{ER}}\, f\big(\varphi^{(3)}_{\text{ER}}(\mathcal{S})\big)\Big],} \tag{26}$$

where $w_{\text{CR}}, w_{\text{EC}}, w_{\text{ER}} > 0$ and $w_{\text{CR}} + w_{\text{EC}} + w_{\text{ER}} = 1$.

Monotone Transform $f(\cdot)$.

$$f(x) \;=\; \frac{\sigma\big(k(x - x_0)\big) - \sigma(-k\,x_0)}{\sigma\big(k(1 - x_0)\big) - \sigma(-k\,x_0)}, \quad \sigma(z) = \frac{1}{1 + e^{-z}}, \tag{27}$$

with steepness $k > 0$ (default 8.0) and inflection $x_0 \in (0,1)$ (default 0.5). The normalization ensures $f(0) = 0$ and $f(1) = 1$. The transform suppresses over-sensitivity to small fluctuations near zero and one while preserving strict monotonicity.

Score Semantics. $\Psi_{\text{CRE}}(\mathcal{S}) = 1$: fully consistent specification—all three $f(\cdot)$ terms in (26) vanish, which requires $\varphi^{(2)}_{\text{CR}} = 0$ (no boundary-seeking relative to π_{unif}), $\bar{\varphi}^{(1)}_{\text{EC}} = 1$ (full critical-state coverage, giving $f(1 - \bar{\varphi}^{(1)}_{\text{EC}}) = f(0) = 0$), and $\varphi^{(3)}_{\text{ER}} = 0$ (perfect reward–utility alignment across environments). $\Psi_{\text{CRE}}(\mathcal{S}) = 0$: maximum inconsistency in all dimensions. The alarm threshold τ_{alarm} (default 0.75) separates the actionable regime ($\Psi_{\text{CRE}} < \tau_{\text{alarm}}$, triggers repair) from the acceptable regime ($\Psi_{\text{CRE}} \geq \tau_{\text{alarm}}$, no action).

5.3 Uncertainty Quantification

Bootstrap confidence intervals are computed for each canonical estimator using $B = 1000$ trajectory resamples with the bias-corrected and accelerated (BCa) method:

$$\text{CI}_{95}(\hat{\varphi}_d) = \big[\hat{\varphi}_d^{(\alpha_1)},\ \hat{\varphi}_d^{(\alpha_2)}\big], \tag{28}$$

where α_1, α_2 are BCa-adjusted quantiles. Uncertainty expansions applied by Stage III Case A (Eq. 23) are propagated through $f(\cdot)$ and accumulated via first-order error propagation to yield a 95 % CI for Ψ_{CRE}.

5.4 LLM-β: Semantic Analysis and Failure Hypothesis

After numerical scoring, LLM-β receives the raw `DiagReport` and the original $\mathcal{S}$. Its sole function is to generate: (1) a natural-language failure mechanism hypothesis mapping numerical findings to causal specification defects; and (2) a priority-ordered list of repair targets. LLM-β does *not* modify any numerical field. A post-processing diff guard verifies this invariant; violation raises `SemanticOverwriteError`.

6 Stage V — Repair Generation

6.1 Repair Space Formalization

Definition 6.1 (Repair Operator)**.** *A* repair operator $\mathcal{R} : \mathcal{S} \to \mathcal{S}'$ *modifies exactly one operator-class of* $\mathcal{S}$ *while leaving all other structural components identical (except for cross-dimensional side-effects that must be explicitly declared in* $\mathcal{R}$.`declare_side_effects`()*).*

The operator vocabulary $\mathcal{V}_{\mathcal{R}}$ is partitioned by target space: *Reward-space* operators: `ReweightTerms`, `InjectPenalty`, `ModifyShaping`, `AdjustClip`; *Environment-space* operators: `ExpandDomainRand`, `InjectCriticalScenes`, `RescaleShiftOp`; *Constraint-adjacent* operators: `AdjustThreshold`, `ScheduleCurriculum`, `ReclassifySeverity`.

Definition 6.2 (Specification Edit Distance)**.** *Let* $G = (V, E)$ *be the typed term graph of* $\mathcal{S}$*, with each node* $v \in V$ *carrying a semantic importance weight* w_v *(e.g., reward weights:* $w_v = 2.0$*; clip bounds:* $w_v = 0.5$*; constraint severity:* $w_v = 3.0$*). The* specification edit distance *is*

$$d_{\text{spec}}(\mathcal{S}, \mathcal{S}') \;=\; \frac{\sum_{v \in V} w_v \cdot \mathbf{1}\big[\mathcal{V}(v, \mathcal{S}) \neq \mathcal{V}(v, \mathcal{S}')\big]}{\sum_{v \in V} w_v}, \tag{29}$$

where $\mathcal{V}(v, \cdot)$ *extracts the value of node* v*.*

6.2 LLM-δ: Repair Generation

LLM-δ receives the `DiagReport` (including semantic analysis), the original $\mathcal{S}$, and the operator vocabulary. It generates $K' = 2K$ candidate proposals (default $K = 3$); half are used as fallbacks after minimality filtering.

Cross-Dimensional Interaction. Every proposal must declare predicted side-effects on non-target dimensions. Declarations are cross-checked against an empirical 3×3 sensitivity matrix M_{sens} (learned from historical repairs). Inconsistencies are flagged but do not cause rejection.

Proposal Ranking. Candidate proposals generated in M7 are ranked before submission to M8 (acceptance verification) to improve expected repair quality and reduce the number of expensive full validation rollouts consumed on low-quality candidates. Ranking proceeds in two stages.

Stage 1 — Lightweight Validation Ranking. Each candidate $\mathcal{S}'_k$ is evaluated on a single fixed-seed environment for N_{rank} episodes (default: $N_{\text{rank}} = 50$), producing a rough score estimate $\hat{\Psi}^{\text{rough}}_{CRE}(\mathcal{S}'_k)$. Proposals are sorted in descending order of rough improvement:

$$\Delta\hat{\Psi}^{\text{rough}}_k \;=\; \hat{\Psi}^{\text{rough}}_{CRE}(\mathcal{S}'_k) \;-\; \hat{\Psi}^{\text{rough}}_{CRE}(\mathcal{S}).$$

This ensures that the most consequential ranking decisions are grounded in observed rollout behaviour rather than model predictions.

Stage 2 — LLM Predicted Improvement as Tiebreaker. When two proposals yield indistinguishable rough scores ($|\Delta\hat{\Psi}_k^{\text{rough}} - \Delta\hat{\Psi}_{k'}^{\text{rough}}| < \epsilon_{\text{rank}}$, default $\epsilon_{\text{rank}} = 0.02$), the LLM-generated field `predicted_delta_psi` is used as a secondary tiebreaker. This field is treated as an informative prior on relative proposal quality, not as a ground-truth ranking signal. It should not be used as the primary ranking criterion, because LLM improvement predictions may be unreliable for proposals involving cross-dimensional interactions.

Configuration. For workflows where trajectory collection is prohibitively expensive, `ranking_mode: "llm_only"` causes Stage 1 to be skipped and all ranking to be based on `predicted_delta_psi`. This mode reduces wall-clock time but may degrade repair loop sample efficiency when LLM predictions are poorly calibrated. It is **not recommended** for safety-critical deployments. The `predicted_delta_psi` field is retained in all modes as a logged diagnostic field; it is always visible in the repair report regardless of the ranking mode in use.

6.3 Minimality Enforcement

Proposals with $d_{\text{spec}}(\mathcal{S}, \mathcal{S}') > \eta_{\text{edit}}$ (default 0.20) are rejected before ranking, ensuring the repair loop does not arbitrarily redesign the specification. Every proposal must include a semantic preservation justification (max 200 tokens), which is later verified by LLM-ε.

7 Stage VI — Acceptance and Validation

7.1 Multi-Criteria Acceptance Protocol

A repair proposal $\mathcal{S}' = \mathcal{R}(\mathcal{S})$ is accepted iff all four criteria are satisfied simultaneously:

$$\text{(C1, Diagnostic):} \quad \Psi_{\text{CRE}}(\mathcal{S}') > \Psi_{\text{CRE}}(\mathcal{S}), \tag{30}$$

$$\text{(C2, Safety):} \quad \max_j P_{\pi^*(\mathcal{S}')}[(s,a) \in K_j] \leq \max_j P_{\pi^*(\mathcal{S})}[(s,a) \in K_j], \tag{31}$$

$$\text{(C3, Utility):} \quad U_{\text{nom}}(\mathcal{S}') \geq U_{\text{nom}}(\mathcal{S}) - \varepsilon_{\text{perf}}, \tag{32}$$

$$\text{(C4, Minimality):} \quad d_{\text{spec}}(\mathcal{S}, \mathcal{S}') \leq \eta_{\text{edit}}. \tag{33}$$

7.2 LLM-ε: Semantic Consistency Verification

LLM-ε receives the original `NLInput` and the de-parsed `NLInput`$'$ (obtained by inverse-mapping $\mathcal{S}'$ to natural language) and produces a semantic consistency score $s_{\text{sem}} \in [0, 1]$ and a binary verdict `intent_preserved`. Acceptance additionally requires $s_{\text{sem}} \geq \tau_{\text{sem}}$ (default 0.80) and `intent_preserved = true`, regardless of C1–C4.

7.3 Accept / Reject Decision and Loop Control

Accept. All criteria (30)–(33) pass and $s_{\text{sem}} \geq \tau_{\text{sem}}$: $\mathcal{S}'$ is adopted, audit record finalized, pipeline exits with `ACCEPT`.

Reject. Any criterion fails: a `RejectionFeedback` record identifies failing criteria and violation magnitudes, and is prepended to the M7 prompt on the next iteration.

Hard Termination. If the loop exceeds $N_{\max}$ iterations (default 5) without acceptance, the pipeline exits with `HARD_REJECT` and generates a human-escalation alert containing the best-seen proposal ($\arg\max_k \Psi_{\text{CRE}}(\mathcal{S}'_k)$).

8 Theoretical Analysis

8.1 Detection Completeness

Assumption 8.1 (Trajectory Coverage)**.** *The dataset $\mathcal{D}_{\mathcal{E}_{\text{tr}}}$ is large enough that $\varphi_{\text{EC}}^{(1,j)}$ concentrates within $\pm\epsilon_c$ of its expectation for all j, with probability $\geq 1-\delta_c$.*

Assumption 8.2 (Utility Oracle Faithfulness)**.** *$U(\pi;\, e) = 1$ iff π successfully completes the task in e; $U(\pi;\, e) \in [0,1]$ is monotone in task completion degree.*

Theorem 8.3 (Framework-Internal Consistency)**.** *Under Assumptions 8.1 and 8.2, and assuming that all three canonical reporters are evaluated without approximation error—specifically, exact boundary computation for $\phi_{CR}^{(2)}$, exact critical-region membership for $\bar{\phi}_{EC}^{(1)}$, and a faithful utility oracle for $\phi_{ER}^{(3)}$—the composite score satisfies*

$$\Psi_{CRE}(\mathcal{S}) < 1$$

if and only if at least one of the following operational canonical reporter conditions *holds:*

$$\phi_{CR}^{(2)}(\mathcal{S}) > 0, \qquad 1 - \bar{\phi}_{EC}^{(1)}(\mathcal{S}) > 0, \qquad \phi_{ER}^{(3)}(\mathcal{S}) > 0.$$

Proof sketch. By Eq. (26),

$$\Psi_{\text{CRE}}(\mathcal{S}) = 1 - \Big[w_{\text{CR}} f\big(\varphi_{\text{CR}}^{(2)}(\mathcal{S})\big) + w_{\text{EC}} f\big(1 - \bar{\varphi}_{\text{EC}}^{(1)}(\mathcal{S})\big) + w_{\text{ER}} f\big(\varphi_{\text{ER}}^{(3)}(\mathcal{S})\big) \Big].$$

Since $w_{\text{CR}}, w_{\text{EC}}, w_{\text{ER}} > 0$ and f is strictly increasing on $[0,1]$ with $f(0) = 0$, the bracketed sum is strictly positive if and only if at least one of the three transformed terms is strictly positive. This is equivalent to at least one of

$$\varphi_{\text{CR}}^{(2)}(\mathcal{S}) > 0, \qquad 1 - \bar{\varphi}_{\text{EC}}^{(1)}(\mathcal{S}) > 0, \qquad \varphi_{\text{ER}}^{(3)}(\mathcal{S}) > 0$$

holding. Therefore $\Psi_{\text{CRE}}(\mathcal{S}) < 1$ if and only if at least one operational canonical reporter condition holds.

The connection to Definitions 1.3–1.7 is one-directional (sufficient) rather than biconditional for all three classes. For Class I, $\phi_{CR}^{(2)} > 0$ is a sufficient operational witness for the boundary-seeking mode discussed in Remark 1.4, but does not capture all manifestations of Definition 1.3. For Class II, any formal undercoverage event $\text{Cov}(K_j, \mathcal{E}_{\text{tr}}) < \delta_j$ implies $\phi_{EC}^{(1,j)} < \delta_j \leq 1$ and hence $1 - \bar{\phi}_{EC}^{(1)} > 0$; the reporter is therefore at least as sensitive as the thresholded definition, though not equivalent to it. For Class III, the formal condition is $\phi_{ER}^{(3)} > \theta_{\text{ER}}$ (Definition 1.7, with $\theta_{\text{ER}} > 0$), while the operational condition is $\phi_{ER}^{(3)} > 0$. Any formal Class III event therefore implies the operational condition, but not conversely: the operational condition is at least as sensitive as the formal definition, not equivalent to it. □ □

Remark 8.4 (Scope of Theorem 8.3)**.** *Theorem 8.3 is a* framework-internal consistency *result. It establishes that Ψ_{CRE} faithfully aggregates the three canonical reporters: whenever any operational canonical reporter condition is active under the ideal-oracle conditions stated above, the composite score is guaranteed to reflect this. It does* ***not*** *constitute a general completeness theorem over all real-world inconsistency failure modes, for the following reasons.*

First, each reporter operates on an approximation of its target definition: $\phi_{CR}^{(2)}$ is a boundary-proximity surrogate for the existential condition in Definition 1.3 (see Remark 1.4); $\bar{\phi}_{EC}^{(1)}$ is a Monte Carlo estimate of set-coverage under a reference policy; and $\phi_{ER}^{(3)}$ presupposes oracle faithfulness, which is a system design commitment rather than a mathematical consequence.

Second, the three reporters collectively cover a specific family of inconsistency manifestations; whether they cover all *practically relevant failure modes is an empirical question that is outside the scope of this theorem. Third, the final score passes through a monotone transform $f(\cdot)$ and a weighted aggregation; while these operations preserve the sign of the inconsistency signal, they introduce additional hyperparameter sensitivity.*

The theorem is therefore most precisely read as: "under the chosen reporters, with ideal oracles, Ψ_{CRE} is equivalent to the framework's own operational definitions"*—a useful internal consistency guarantee, but not a claim about detection completeness with respect to the ground-truth inconsistency space.*

8.2 Algorithm-Agnosticity

Theorem 8.5 (Algorithm-Agnosticity of the Score Functional)**.** *For any fixed policy π and specification $\mathcal{S}$, the score $\Psi_{\mathrm{CRE}}(\mathcal{S};\pi)$ as defined in (26) is invariant to the choice of RL optimizer $\mathcal{O}$ and critic architecture ϕ_c.*

Proof. All three canonical reporters $\varphi^{(2)}_{\mathrm{CR}}$, $\bar{\varphi}^{(1)}_{\mathrm{EC}}$, $\varphi^{(3)}_{\mathrm{ER}}$ are defined solely in terms of trajectory statistics under π, the environments $\mathcal{E}_{\mathrm{tr}}/\mathcal{E}_{\mathrm{dep}}$, and the constraint predicates $\{\mathcal{O}_j\}$—none of which references the optimizer $\mathcal{O}$ or critic ϕ_c. Therefore $\Psi_{\mathrm{CRE}}(\mathcal{S};\pi)$ is invariant to these choices. □ □

Remark 8.6 (Practical Scope of Algorithm-Agnosticity)**.** *Theorem 8.5 establishes that the* scoring functional $\Psi_{CRE}(\mathcal{S};\pi)$ *is algorithm-agnostic: for any fixed policy input π, the score depends only on π's behavioral trajectory distribution and the specification $\mathcal{S}$, not on the optimizer or critic architecture that produced π. This property holds by construction, since Ψ_{CRE} is defined purely in terms of π's rollout statistics.*

This result should ***not*** *be interpreted as implying that the overall training pipeline is algorithm-agnostic. In practice, the policy π evaluated by Ψ_{CRE} is itself the output of an optimizer-dependent training process. The reward-optimal policy π^* is generally unobservable, non-unique, and approximated differently by different optimizers. Consequently, two pipelines that differ only in their optimizer will typically produce distinct policies $\pi_1 \neq \pi_2$, yielding distinct scores $\Psi_{CRE}(\mathcal{S};\pi_1) \neq \Psi_{CRE}(\mathcal{S};\pi_2)$. The present theorem provides no guarantees about this optimizer-induced variance in observed scores.*

8.3 Repair Loop Convergence

Theorem 8.7 (Repair Loop Termination Guarantee)**.** *Let $\mathcal{B}(\mathcal{S},\eta)$ denote the edit ball of radius η around specification $\mathcal{S}$. Suppose:*

(i) $|\mathcal{B}(\mathcal{S},\eta)| < \infty$*;*

(ii) there exists at least one proposal $\mathcal{S}' \in \mathcal{B}(\mathcal{S},\eta)$ satisfying all acceptance criteria C1–C4*;*

(iii) the LLM proposal module assigns probability at least $p_{\min} > 0$ to every element of $\mathcal{B}(\mathcal{S},\eta)$ on each iteration.

Then for any $\delta \in (0,1)$, the repair loop finds an acceptable proposal within

$$N \;\leq\; \left\lceil \frac{\log \delta}{\log(1-p_{\min})} \right\rceil$$

iterations with probability at least $1-\delta$.

Remark 8.8 (Interpretation and Limitations of Theorem 8.7)**.** *This theorem provides a* ***finite-time termination probability bound*** *for the repair loop: if a valid fix exists within the edit ball and the LLM has nonzero probability of generating it, the loop will terminate within a computable iteration budget. The result is structurally equivalent to a random-search hitting-time bound over a finite set.*

The theorem does ***not*** *guarantee convergence in repair quality. The accepted proposal is the first one to pass criteria C1–C4, not necessarily the highest-quality feasible repair. The ranking mechanism in M7 (Section 6) is intended to improve the expected quality of the accepted proposal within the termination budget, but its quality improvement is not formally bounded here. Additionally, the three assumptions—particularly (i) finiteness of the edit ball and (iii) the uniform positive-probability condition on the LLM—are idealizations; their practical validity depends on the tokenization of the specification space and the LLM's generation distribution, which are domain-dependent. Empirical validation of termination rates across benchmark tasks is reported in Section* **??**.

8.4 Computational Complexity

Let T be the trajectory length, N_{traj} the number of trajectories, m the number of constraints, and $n = |\mathcal{E}_{\text{dep}}|$ the number of deployment environments. Estimation (M2) costs $\mathcal{O}(N_{\text{traj}} \cdot T \cdot (m+n))$. Aggregation (M5) is $\mathcal{O}(1)$. Enhanced estimation (M3) costs $\mathcal{O}(N_g \cdot d^2)$ where N_g is the gradient sample count and $d = |\theta|$. The LLM call budget is ≤ 5 calls per iteration, each bounded by $\mathcal{O}(L_{\max})$ tokens. Total pipeline cost is $\mathcal{O}(N_{\max} \cdot N_{\text{traj}} \cdot T \cdot m)$ plus $\mathcal{O}(N_{\max} \cdot L_{\max})$ for LLM inference.

Part II

Agent Developer Handbook

Handbook Purpose and Scope. Part II is an implementation-level specification for agents building CRE pipeline components. Every

0.0.1

is a **self-contained execution specification**: it states the exact function signature, a typed input/output contract, all configurable parameters with their defaults, the complete set of error codes the function may raise, and a minimum test battery sufficient to verify correctness. No knowledge beyond Part I and Part II is required for implementation. Part I equation numbers are cited for formula derivation; operational formulas are restated inline.

Document Conventions.

- **Type system.** Signatures use Python 3.10 type-hint syntax. `T | None` is equivalent to `Optional[T]`. Array shapes are written as `ndarray[shape, dtype]`; e.g. `ndarray[(N,d_S), float64]`.
- **Immutability.** Every `SpecS` argument is read-only. Functions that produce a modified specification return a new object; the original is never mutated in-place.
- **Floating-point range.** All scalar estimators produce `float64`. Values guaranteed to lie in $[0, 1]$ are post-clipped to [0.0, 1.0] by the returning function before the value is stored.

- **Error taxonomy.** All errors are instances of `CREError` (base class) or a registered subclass (Section 11). Error codes are uppercase underscore strings bound to the constant registry in Section 11.
- **LLM retry semantics.** Every LLM call (`M1`, `M4`, `M6`, `M7`, `M8`) wraps the model invocation in a retry-with-exponential-backoff loop governed by `CFG.llm_retry_limit`. On exhaustion, the function raises `LLM_RETRY_EXHAUSTED`.
- **Audit trail.** Every function appends a typed record to the shared `AuditTrail` object (schema: Section 10.1.2). Functions that skip this step are marked **[no audit]**.
- **Singleton config.** The global configuration object `CFG` (Section 1.3) is the single source of truth for all defaults. Individual function parameters *override* the corresponding `CFG` field when supplied explicitly.

1 Architecture and Global Definitions

1.1 Module Registry and Pipeline Topology

Table 3 lists all eight pipeline modules, their stage assignment, LLM/deterministic classification, and the section in Part II where their API is specified.

Table 3: CRE module registry.

ID	Alias	Stage	Type	Role	Spec
M1	`LLMa`	I	LLM	NL parsing & ambiguity detection	§2
M2	—	II	Det.	Primary estimator computation	§3
M3	—	II	Det.	Enhanced estimator computation	§4
M4	`LLMb`	IV	LLM	Semantic analysis & repair-target ranking	§7
M5	—	IV	Det.	Composite score aggregation	§6
M6	`LLMg`	III	LLM	Temporal hypothesis (Case A only)	§5
M7	`LLMd`	V	LLM	Repair proposal generation & ranking	§8
M8	`LLMe`	VI	LLM	Semantic consistency verification	§9

Execution order. `M1` → `M2` → `M3` → [`Stage III` : `DP(M6)`] → `M5` → `M4` → [`loop` : `M7` → `M8`]. Brackets denote conditional execution. M3, M6, and the Stage III Discrepancy Protocol are skipped if `CFG.enhanced_estimators_enabled = False`.

1.2 Core Data Schemas

The following pseudo-typed structures define the canonical data exchange format between modules. All field names used in function signatures below refer to these schemas.

NLInput.

```
@dataclass
class NLInput:
    r_desc: str          # reward description (required, non-empty)
    c_desc: str          # constraint description (required, non-empty)
    e_desc: str          # environment description (required, non-empty)
    metadata: dict       # free-form, passed through unchanged
```

Constraint.

```
@dataclass
class Constraint:
```

```
    k_id:    str         # unique constraint ID, e.g. "C1"
    predicate: Callable[[State, Action], int]  # O_j: S×A -> {0,1}
    sigma:   Literal["hard","soft","info"]
    scope:   Literal["instantaneous","episodic","cumulative"]
    zeta:    float | None  # violation tolerance; required iff sigma=="soft"
    lam:     float | None  # penalty weight; optional for soft constraints
    delta:   float         # coverage threshold delta_j > 0; default CFG.delta_j_default
```

RewardDAG.

```
@dataclass
class RewardDAG:
    nodes: list[RewardTerm]  # each term r_k with weight w_k, clip [c_k^-, c_k^+]
    edges: list[tuple[str,str]]  # (parent_id, child_id) shaping deps
    # composite: R(s,a) = sum_k w_k * clip(r_k(s,a), c_k^-, c_k^+)
```

SpecS.

```
@dataclass(frozen=True)   # immutable
class SpecS:
    spec_id:  str          # UUID assigned by M1 at parse time
    E_tr:     TrainingEnvDistribution
    E_dep:    list[Environment]   # [e_0, e_1, ..., e_n]; e_0 is nominal
    R:        RewardDAG
    C:        list[Constraint]    # length m
    Pi:       PolicyClass
    version:  int = 0             # incremented on each repair
```

AmbiguityFlag.

```
@dataclass
class AmbiguityFlag:
    flag_type:    Literal["UNDEFINED_THRESHOLD","MISSING_ENV_SCOPE",
                          "REWARD_UNDERSPECIFIED","CONSTRAINT_OVERLAP",
                          "TEMPORAL_AMBIGUITY"]
    location:     str      # dot-separated JSON path within SpecS
    impact_score: float    # in [0,1]; estimated |DeltaPsiCRE| reduction
```

DiagReport.

```
@dataclass
class DiagReport:
    spec_id:       str
    timestamp:     str            # ISO 8601
    # --- canonical reporters (always present after M2) ---
    phi_cr2:       float          # in [0,1]
    phi_ec_bar:    float          # in [0,1]
    phi_er3:       float          # in [0,1]
    # --- supplementary reporters (None if computation skipped) ---
    phi_cr1:       float | None
    phi_ec2:       float | None
    phi_ec3:       float | None
```

```
    phi_ec_per_j:  list[float] | None   # per-constraint coverage
    phi_er1:       float | None
    phi_er2:       float | None
    # --- enhanced reporters (None if M3 not run) ---
    kappa_cr:      float | None
    gamma_ec:      float | None
    delta_er:      float | None
    # --- composite score ---
    psi_cre:       float          # in [0,1]
    ci_95:         dict[str, tuple[float,float]]  # key = reporter name
    # --- metadata ---
    flags:         list[str]      # error / warning codes
    discrepancy:   list[DiscrepancyRecord] | None
    failure_hypothesis: str | None   # set by M4
    repair_targets:     list[str] | None  # set by M4
```

RepairProposal.

```
@dataclass
class RepairProposal:
    proposal_id:          str    # UUID
    spec_prime:           SpecS
    operator_class:       str    # from V_R vocabulary
    declared_side_effects: dict[str, str]   # dim -> description
    semantic_justification: str  # max 200 tokens
    predicted_delta_psi:  float | None   # LLM-predicted improvement
    rough_delta_psi:      float | None   # set by M7.rank_proposals Stage 1
```

AcceptanceVerdict.

```
@dataclass
class AcceptanceVerdict:
    accepted:      bool
    c1_pass:       bool   # PsiCRE improvement
    c2_pass:       bool   # safety non-regression
    c3_pass:       bool   # utility non-regression
    c4_pass:       bool   # minimality
    s_sem:         float  # in [0,1]
    intent_preserved: bool
    rejection_feedback: str | None  # populated on rejection
```

1.3 Global Configuration Constants

`CFG` is a singleton loaded at pipeline start. All fields may be overridden by environment variables or a YAML config file. Table 4 lists every field, its type, default, and the Part I symbol it binds.

Table 4: Global configuration constants (`CFG.*`).

Field	Type	Default	Part I Symbol
Ambiguity Detection (M1)			
`eta_flag`	float	0.15	η_{flag}
`llm_retry_limit`	int	3	—
`delta_j_default`	float	0.50	δ_j (default)
Estimators (M2/M3)			
`n_ref_traj`	int	500	N_{traj}
`n_mc_kj`	int	1000	Monte Carlo samples for $\lvert K_j \rvert$
`critical_region_radius`	float	0.10	r_j
`ec_aggregation_mode`	str	`"mean"`	—
`eps_stab`	float	1e-6	ϵ_{stab}
`enhanced_estimators_enabled`	bool	True	—
Discrepancy Protocol (Stage III)			
`eta_disc`	float	0.15	η_{disc}
`tau_low`	float	0.30	τ_{low}
`tau_high`	float	0.70	τ_{high}
`lambda_expand`	float	0.50	λ
`n_rechk`	int	1	N_{rechk}
Composite Scoring (M5)			
`f_steepness`	float	8.0	k
`f_inflection`	float	0.50	x_0
`w_cr`	float	0.333	w_{CR}
`w_ec`	float	0.333	w_{EC}
`w_er`	float	0.334	w_{ER}
`b_bootstrap`	int	1000	B
`k_detect`	float	1.5	k_d
`k_alarm`	float	3.0	k_a
`tau_alarm_psi`	float	0.75	τ_{alarm} (PsiCRE)
Repair Generation (M7)			
`k_proposals`	int	3	K
`eta_edit`	float	0.20	η_{edit}
`n_rank_episodes`	int	50	N_{rank}
`eps_rank`	float	0.02	ϵ_{rank}
`ranking_mode`	str	`"rollout_first"`	—
Acceptance (M8)			
`tau_sem`	float	0.80	τ_{sem}
`eps_perf`	float	0.05	ϵ_{perf}
`n_max_iterations`	int	5	$N_{\max}$

Weight constraint enforcement. At pipeline start, the orchestrator verifies `w_cr` + `w_ec` + `w_er` = 1.0 (within 10^{-9} tolerance) and all three weights are strictly positive. Violation raises `CONFIG_WEIGHT_SUM_ERROR` before any module runs.

2 Stage I — Module M1: Specification Parsing

2.1 Module Overview

M1 (`LLMa`) is the sole entry point for natural-language input. It produces an immutable `SpecS` and a list of `AmbiguityFlag` objects. M1 must complete without error before any downstream module runs. The prompt template $\mathcal{P}_\alpha$ referenced below is defined in Appendix B.1 of Part I.

2.1.1 M1.parse_nl_input

Signature.

```
def M1.parse_nl_input(
    nl: NLInput,
    prompt_template: str = PROMPT_ALPHA,
    llm_backend:     str = "default",
    retry_limit:     int = CFG.llm_retry_limit,   # default 3
    null_fill:       bool = True,
) -> tuple[SpecS, list[AmbiguityFlag]]
```

Input Contract.

Field	Type	Required	Constraint
`nl.r_desc`	str	R	non-empty
`nl.c_desc`	str	R	non-empty
`nl.e_desc`	str	R	non-empty
`prompt_template`	str	O	valid Jinja2 template
`retry_limit`	int	O	≥ 1
`null_fill`	bool	O	if `True`: unresolvable fields → typed `null`

Output Contract. Returns `(spec, flags)` where:

- `spec` is a valid, frozen `SpecS` with `spec.spec_id` set to a fresh UUID.
- `flags` is a (possibly empty) list of `AmbiguityFlag`; all `impact_score` values $\in [0, 1]$.
- Every field of `spec` is either fully populated or set to a typed `null` sentinel (never a bare Python `None` for required fields).
- `spec.version = 0` (initial parse).

Error Codes.

Code	Trigger	Recovery
`SPEC_PARSE_FAILURE`	LLM output is not valid JSON after `retry_limit` attempts	Raise; escalate to human
`NULL_REWARD`	`spec.R.nodes` is empty after fill	Raise; input `r_desc` is too u specified
`EMPTY_CONSTRAINT_SET`	`spec.C` is empty after fill	Warn only; downstream without safety constraints
`EMPTY_ENV_SET`	`len(spec.E_dep)` < 1	Raise; at least e_0 (nominal required
`LLM_RETRY_EXHAUSTED`	All retry attempts failed	Raise `CREError`; attach raw output to record

Test Standards.

T1. Round-trip parse. Feed a fully-specified NLInput with one reward term, one hard constraint, two environments. Assert `len(spec.C)==1`, `spec.C[0].sigma=="hard"`, `len(spec.E_dep)==2`, `flags==[]` (no ambiguities).

T2. Null-fill activation. Feed an NLInput with `c_desc=""` (empty constraint description). With `null_fill=True`: assert `spec.C==[]` and `EMPTY_CONSTRAINT_SET` in `flags`. With `null_fill=False`: assert `SPEC_PARSE_FAILURE` raised.

T3. Ambiguity flag generation. Feed an NLInput describing a reward with no explicit weights. Assert at least one flag with `flag_type=="REWARD_UNDERSPECIFIED"` and `impact_score` $\in (0, 1]$.

T4. Retry semantics. Mock the LLM to fail twice then succeed. Assert result is valid `(spec, flags)` and no error is raised when `retry_limit=3`.

T5. Version field. Assert `spec.version==0` for all fresh parses.

T6. Immutability. Attempt to assign `spec.version=1` after return. Assert `FrozenInstanceError` (or equivalent) is raised.

2.1.2 M1.detect_and_escalate_ambiguities

Signature.

```
def M1.detect_and_escalate_ambiguities(
    flags:          list[AmbiguityFlag],
    eta_flag:       float = CFG.eta_flag,    # default 0.15
    halt_on_high:   bool  = True,
) -> list[AmbiguityFlag]
```

Input Contract. `flags` is the raw list returned by `M1.parse_nl_input`. `eta_flag` is the escalation threshold for `impact_score`. `halt_on_high`: if `True`, any flag with `impact_score > eta_flag` that cannot be auto-resolved causes the function to raise `AMBIGUITY_UNRESOLVABLE`.
Output Contract. Returns the subset of `flags` with `impact_score` $\leq$ `eta_flag` (low-impact flags retained for logging). High-impact flags are either auto-resolved (their corrected values are written back to a staging patch dict) or raise `AMBIGUITY_UNRESOLVABLE`.
Error Codes.

Code	Trigger	Recovery
`AMBIGUITY_UNRESOLVABLE`	$\exists$ flag: `impact_score` $>$ `eta_flag` and no auto-resolution available	Raise; human revi pipeline proceeds

Test Standards.

T1. Threshold boundary. Construct a flag list with `impact_score=0.15` (at threshold). Assert the flag is *not* escalated (strict inequality: only `> eta_flag` triggers escalation).

T2. Escalation halt. Construct a flag with `impact_score=0.20`. With `halt_on_high=True`: assert `AMBIGUITY_UNRESOLVABLE` raised. With `halt_on_high=False`: assert function returns without error and the flag appears in returned list.

T3. Empty list passthrough. Input `flags==[]`. Assert return is `[]`.

2.1.3 M1.run_symbolic_precheck

Signature.

```
def M1.run_symbolic_precheck(
    spec: SpecS,
) -> list[str]   # returns list of warning/error codes; raises on fatal
```

Input Contract. `spec` is the frozen `SpecS` returned by `M1.parse_nl_input` after ambiguity resolution. No additional parameters.
Output Contract. Returns a (possibly empty) list of non-fatal warning codes. Raises `PRECHECK_FATAL` for any of three fatal conditions:

1. Type incompatibility: some reward term or constraint predicate references a state/action subspace not supported by the others.
2. Domain boundary violation: the support of R does not intersect the interior of $\mathcal{S} \setminus \bigcup_j K_j$ (trivial repair exists—pipeline should not proceed).
3. Coverage pre-bound failure: $|\mathcal{F}|$ provides fewer than one scene family per hard constraint.

Error Codes.

Code	Trigger	Recovery
`PRECHECK_TYPE_MISMATCH`	Condition 1 above (fatal)	Raise
`PRECHECK_DOMAIN_TRIVIAL`	Condition 2 above (fatal)	Raise; apply trivial repair directly
`PRECHECK_COVERAGE_INSUFFICIENT`	Condition 3 above (fatal)	Raise
`PRECHECK_SOFT_NO_TOLERANCE`	Soft constraint with `zeta=None`	Return as warning code

Test Standards.

T1. Clean spec passes. Provide a well-formed `SpecS` with compatible subspaces and at least one scene family per hard constraint. Assert return is `[]`.

T2. Domain trivial. Provide a spec where $K_j = \mathcal{S}$ for some j (entire state space is violating). Assert `PRECHECK_DOMAIN_TRIVIAL` raised.

T3. Coverage pre-bound. Provide a spec with one hard constraint but zero scene families (`spec.E_tr.scene_families=[]`). Assert `PRECHECK_COVERAGE_INSUFFICIENT` raised.

T4. Soft with no tolerance. Provide a spec with one soft constraint where `zeta=None`. Assert `PRECHECK_SOFT_NO_TOLERANCE` in returned list (no exception raised).

3 Stage II — Module M2: Primary Estimators

3.1 Module Overview

M2 computes all behavioral estimators listed in Part I §3.3–§3.5. All M2 functions are deterministic given fixed trajectory data. M2 functions may not write to canonical reporter slots if they are computing supplementary estimators; the orchestrator enforces this via the `@supplementary_only` decorator. Any code path that writes a supplementary or enhanced value into a canonical slot `phi_cr2`, `phi_ec_bar`, or `phi_er3` of `DiagReport` raises `CANONICAL_VIOLATION_ERROR`.

3.2 C–R Dimension

3.2.1 M2.compute_phi_cr1

Signature.

```
@supplementary_only
def M2.compute_phi_cr1(
    trajectories: list[Trajectory],   # sampled under pi_star in E_tr
    spec:         SpecS,
) -> float | None
```

Formula (Part I Eq. (10)).

$$\phi_{CR}^{(1)} = \frac{\mathrm{Cov}_{\tau\sim\pi^*}\big(\sum_t \gamma^t R(s_t,a_t),\ \mathbf{1}[\exists\, t : (s_t,a_t) \in \bigcup_j K_j]\big)}{\sigma_R \cdot \sigma_C}.$$

Input Contract.

Field	Type	Req.	Constraint
`trajectories`	`list[Trajectory]`	R	≥ 30 trajectories; each trajectory ≥ 1 step
`spec.C`	`list[Constraint]`	R	non-empty

Output Contract. Returns a `float` in $[-1, 1]$ (Pearson correlation). Returns `None` and appends `DEGENERATE_CR1_VAR` to `DiagReport.flags` when $\sigma_R \cdot \sigma_C = 0$. Positive value indicates high-reward trajectories are disproportionately associated with constraint violations.

Error Codes.

Code	Trigger	Recovery
`DEGENERATE_CR1_VAR`	$\sigma_R = 0$ or $\sigma_C = 0$ in trajectory data	Set return to `None`; attach flag; do not raise
`INSUFFICIENT_TRAJECTORIES`	`len(trajectories) < 30`	Raise `CREError`

Test Standards.

T1. Positive correlation. Construct 100 synthetic trajectories where high-return episodes always visit K_j. Assert `phi_cr1 > 0.5`.

T2. Zero correlation. Construct trajectories with uncorrelated returns and violations. Assert `|phi_cr1| < 0.1`.

T3. Degenerate variance. Construct trajectories with constant return (no variance in R). Assert return is `None` and `DEGENERATE_CR1_VAR` in flags.

T4. Result range. Assert $-1 \leq$ `phi_cr1` ≤ 1 for all non-None returns.

3.2.2 M2.compute_phi_cr2

Signature.

```
def M2.compute_phi_cr2(
    traj_pistar: list[Trajectory],   # under pi_star in E_tr
    traj_unif:   list[Trajectory],   # under pi_unif in E_tr
    spec:        SpecS,
) -> float
```

Formula (Part I Eq. (11)). Let D^* = mean minimum boundary distance under π^*, D^{unif} = mean minimum boundary distance under π_{unif}:

$$\phi_{CR}^{(2)} = \max\Big(0,\ 1 - D^*/D^{\text{unif}}\Big), \qquad \partial K_j = \{s : \mathcal{O}_j(s,a) = 1 \text{ for some } a\}.$$

Boundary ∂K_j is approximated by nearest-neighbour lookup.

Input Contract.

Field	Type	Req.	Constraint
`traj_pistar`	`list[Trajectory]`	R	≥ 30 trajectories under π^*
`traj_unif`	`list[Trajectory]`	R	≥ 30 trajectories under π_{unif}
`spec.C`	`list[Constraint]`	R	$\geq$ 1 hard constraint (`sigma=="hard"`)

Output Contract. Returns a `float` in $[0,1]$ (post-clipped by `max(0,·)`). $\phi_{CR}^{(2)} = 0$: π^* maintains at least as great expected boundary clearance as π_{unif}. $\phi_{CR}^{(2)} = 1$: π^* achieves zero expected boundary distance.

Error Codes.

Code	Trigger	Recovery
`BOUNDARY_BASELINE_DEGENERATE`	$D^{\text{unif}} = 0$ (random policy has zero boundary clearance)	Raise `CREError`; spec i
`NO_HARD_CONSTRAINTS`	`spec.C` has no `sigma=="hard"` entries	Raise `CREError`
`INSUFFICIENT_TRAJECTORIES`	Either trajectory list has < 30 entries	Raise `CREError`

Test Standards.

T1. Policy safer than uniform. Construct $D^* = 2.0$, $D^{\text{unif}} = 1.0$ (pistar twice as far from boundary). Assert `phi_cr2 == 0.0` (truncated by `max(0,·)`).

T2. Policy at boundary. Construct $D^* = 0.0$, $D^{\text{unif}} = 1.0$. Assert `phi_cr2 == 1.0`.
T3. Policy equals uniform. Construct $D^* = D^{\text{unif}} = 1.0$. Assert `phi_cr2 == 0.0` (ratio $= 1$, $1 - 1 = 0$).
T4. Partial boundary-seeking. Construct $D^* = 0.6$, $D^{\text{unif}} = 1.0$. Assert `phi_cr2` ≈ 0.40 (± 0.01).
T5. Degenerate baseline. Set $D^{\text{unif}} = 0$. Assert `BOUNDARY_BASELINE_DEGENERATE` raised.
T6. Range guarantee. Assert $0.0 \leq$ `phi_cr2` ≤ 1.0 for all valid inputs.

3.3 E–C Dimension

3.3.1 M2.compute_phi_ec_j

Signature.

```
def M2.compute_phi_ec_j(
    constraint:   Constraint,     # single C_j with sigma=="hard"
    dataset:      list[Trajectory],  # D_{E_tr} under pi_ref
    n_mc_kj:      int   = CFG.n_mc_kj,              # default 1000
    radius:       float = CFG.critical_region_radius, # default 0.10
) -> float
```

Formula (Part I Eq. (12)).

$$\phi_{EC}^{(1,j)} = \frac{\left|\{s \in K_j \mid \exists \tau \in \mathcal{D}_{\mathcal{E}_{\text{tr}}} : s \in \tau\}\right|}{|K_j|},$$

where $|K_j|$ is estimated by Monte Carlo sampling of the constraint predicate using a KD-tree with lookup radius `radius`.
Input Contract.

Field	Type	Req.	Constraint
`constraint.sigma`	str	R	must equal `"hard"`
`dataset`	`list[Trajectory]`	R	≥ 1 trajectory; reference policy rollouts
`n_mc_kj`	int	O	≥ 100
`radius`	float	O	> 0

Output Contract. Returns a `float` in $[0, 1]$: fraction of the critical region K_j visited by at least one trajectory in `dataset`. 1.0 = full coverage; 0.0 = no state in K_j was ever visited.
Error Codes.

Code	Trigger	Recovery
`WRONG_SEVERITY`	`constraint.sigma` $\neq$ `"hard"`	Raise `CREError`; caller should filter by severity
`EMPTY_KJ_ESTIMATE`	MC sampling yields 0 points in K_j	Return `0.0`; append `KJ_EMPTY_WARN` flag

Test Standards.
T1. Full coverage. Provide trajectories that densely visit K_j. Assert `phi_ec_j` > 0.95.
T2. Zero coverage. Provide trajectories that never enter K_j. Assert `phi_ec_j == 0.0`.
T3. Wrong severity. Pass a constraint with `sigma=="soft"`. Assert `WRONG_SEVERITY` raised.
T4. Radius sensitivity. Run with `radius=0.01` and `radius=0.50` on the same data. Assert larger radius yields $\geq$ coverage (monotone in radius).

3.3.2 M2.compute_phi_ec_bar

Signature.

```
def M2.compute_phi_ec_bar(
    per_j_scores:       list[float],   # phi^{(1,j)}_{EC} for each hard C_j
    aggregation_mode:   Literal["mean","min"] = CFG.ec_aggregation_mode,
) -> float
```

Formula (Part I Eq. (13)).

$$\bar{\phi}_{EC}^{(1)} = \begin{cases} \frac{1}{m_{\text{hard}}} \sum_j \phi_{EC}^{(1,j)} & \text{if mode} = \texttt{"mean"} \\ \min_j \phi_{EC}^{(1,j)} & \text{if mode} = \texttt{"min"} \end{cases}$$

Input Contract.

Field	Type	Req.	Constraint
`per_j_scores`	`list[float]`	R	non-empty; each value $\in [0,1]$
`aggregation_mode`	str	O	`"mean"` or `"min"`

Output Contract. Returns a `float` in $[0,1]$. This value is the canonical E–C reporter; it is stored in `DiagReport.phi_ec_bar`. The complementary undercoverage term $1 - \bar{\phi}_{EC}^{(1)}$ is passed to `M5.compute_psi_cre` as the E–C signal.

Error Codes.

Code	Trigger	Recovery
`EMPTY_HARD_CONSTRAINTS`	`per_j_scores` is empty	Raise `CREError`
`INVALID_AGGREGATION_MODE`	mode not in `{"mean","min"}`	Raise `CREError`

Test Standards.

T1. Mean mode. Input `[0.4, 0.6, 0.8]`, mode `"mean"`. Assert result ≈ 0.600 ($\pm 10^{-9}$).

T2. Min mode. Input `[0.4, 0.6, 0.8]`, mode `"min"`. Assert result $= 0.4$.

T3. Min masking property. Input `[0.0, 1.0, 1.0]`, mode `"mean"`. Assert result ≈ 0.667 (zero-coverage constraint masked). Mode `"min"`: assert result $= 0.0$ (not masked).

T4. Single constraint. Input `[0.73]`, both modes. Assert result $= 0.73$.

3.3.3 M2.compute_phi_ec2

Signature.

```
@supplementary_only
def M2.compute_phi_ec2(
    trajectories: list[Trajectory],   # under pi_star in E_tr
    spec:         SpecS,
) -> float
```

Formula (Part I Eq. (14)). $\phi_{EC}^{(2)} = \mathbb{E}_{\tau \sim \pi^*, \mathcal{E}_{\text{tr}}}[\frac{1}{T} \sum_{t=0}^{T-1} \mathbf{1}[(s_t, a_t) \in \bigcup_j K_j]]$. Mean fraction of time steps that lie in any constraint region.

Output Contract. Returns `float` in $[0,1]$. High values indicate the optimal policy frequently violates constraints. This is a supplementary reporter and must not be written to `phi_ec_bar` in `DiagReport`.

Error Codes.

Code	Trigger	Recovery
`EMPTY_TRAJECTORY_LIST`	`len(trajectories)==0`	Raise `CREError`

Test Standards.

T1. Zero violation. All states in trajectories lie outside $\bigcup_j K_j$. Assert `phi_ec2 == 0.0`.
T2. Full violation. All steps lie inside some K_j. Assert `phi_ec2 == 1.0`.
T3. Range guarantee. Assert $0.0 \leq$ `phi_ec2` ≤ 1.0.

3.3.4 M2.compute_phi_ec3

Signature.

```
@supplementary_only
def M2.compute_phi_ec3(
    trajectories: list[Trajectory],
    spec:         SpecS,
) -> float
```

Formula (Part I Eq. (15)). $\phi_{EC}^{(3)} = H(\{p_j\}_{j=1}^m)/\log m$ where $p_j = \mathbb{E}_\tau[\mathbf{1}[(s_t, a_t) \in K_j]]$ and H is Shannon entropy. Measures distributional uniformity of constraint activations.
Output Contract. Returns `float` in $[0, 1]$. 1.0: all constraints activated equally. 0.0: only one constraint ever activated. Returns `1.0` by convention when $m = 1$ (degenerate but valid).
Error Codes.

Code	Trigger	Recovery
`SINGLE_CONSTRAINT_DIVERSITY`	$m = 1$	Return `1.0`; append informational flag
`ZERO_CONSTRAINT_ACTIVATIONS`	All $p_j = 0$	Return `0.0`; no constraints were ever activated

Test Standards.

T1. Uniform activation. Three constraints, each activated in exactly 1/3 of steps. Assert `phi_ec3` ≈ 1.0.
T2. Single constraint activated. Three constraints; only C_1 ever activated. Assert `phi_ec3` ≈ 0.0.
T3. Single constraint in spec. $m = 1$. Assert return $= 1.0$.

3.4 E–R Dimension

3.4.1 M2.compute_phi_er1

Signature.

```
@supplementary_only
def M2.compute_phi_er1(
    utility_scores: dict[str, float],  # env_id -> U(pi_star; e_i)
    spec:           SpecS,
) -> float
```

Formula (Part I Eq. (16)). $\phi_{ER}^{(1)} = U_{\text{nom}}(\pi^*) - \frac{1}{n}\sum_{i=1}^n U(\pi^*; e_i)$, where the nominal utility is $U(\pi^*; e_0)$ and the mean is taken over shifted environments $e_1, \ldots, e_n$.
Input Contract. `utility_scores` must contain a key for the nominal environment ID (the first element of `spec.E_dep`).
Output Contract. Returns `float` in $[-1, 1]$. Positive values indicate performance degradation under distribution shift.
Error Codes.

Code	Trigger	Recovery
`MISSING_NOMINAL_UTILITY`	`utility_scores` lacks key for e_0	Raise `CREError`
`NO_SHIFTED_ENVS`	`len(spec.E_dep)==1` (only nominal)	Return `0.0`; append `SINGLE_ENV_WARN` flag

Test Standards.

T1. No gap. $U(e_0) = 0.8$, $U(e_1) = U(e_2) = 0.8$. Assert `phi_er1 == 0.0`.
T2. Large gap. $U(e_0) = 1.0$, $U(e_1) = 0.2$, $U(e_2) = 0.2$. Assert `phi_er1` $\approx$ `0.8`.
T3. Missing nominal. Remove e_0 key. Assert `MISSING_NOMINAL_UTILITY` raised.

3.4.2 M2.compute_phi_er2

Signature.

```
@supplementary_only
def M2.compute_phi_er2(
    reward_scores: dict[str, float],  # env_id -> R_norm(pi_star; e_i)
    spec:          SpecS,
) -> float
```

Formula (Part I Eq. (17)). $\phi_{ER}^{(2)} = R_{\text{norm}}(\pi^*; e_0) - \frac{1}{n}\sum_{i=1}^{n} R_{\text{norm}}(\pi^*; e_i)$, where each $R_{\text{norm}} \in [0, 1]$.
Input Contract. Same structure as `M2.compute_phi_er1` with normalized reward scores replacing utility scores. All values must be in $[0, 1]$.
Output Contract. Returns `float` in $[-1, 1]$. Positive values indicate reward degradation under shift.
Error Codes. Same as `M2.compute_phi_er1`. Additionally: `REWARD_OUT_OF_RANGE` if any score $\notin [0, 1]$.
Test Standards.

T1. No degradation. All scores $= 0.7$. Assert `phi_er2 == 0.0`.
T2. Out-of-range input. Score $= 1.5$. Assert `REWARD_OUT_OF_RANGE` raised.

3.4.3 M2.compute_phi_er3

Signature.

```
def M2.compute_phi_er3(
    reward_scores:  dict[str, float],  # env_id -> R_norm in [0,1]
    utility_scores: dict[str, float],  # env_id -> U in [0,1]
    spec:           SpecS,
) -> float
```

Formula (Part I Eq. (18)).

$$\phi_{ER}^{(3)} = \frac{1}{2}\left(1 - \frac{\sum_{i=0}^{n}(R_i - \bar{R})(U_i - \bar{U})}{\sqrt{\sum_i (R_i - \bar{R})^2 \cdot \sum_i (U_i - \bar{U})^2}}\right),$$

where $\bar{R}, \bar{U}$ are cross-environment sample means over $\{e_0, \ldots, e_n\}$.
Input Contract.

Field	Type	Req.	Constraint
`reward_scores`	`dict[str,float]`	R	keys = all env IDs in `spec.E_dep`; values $\in [0,1]$
`utility_scores`	`dict[str,float]`	R	same keys; values $\in [0,1]$; must be constructed independently of R (Assumptions 3.2–3.4)

Output Contract. Returns `float` in $[0,1]$: 0.0 = perfect alignment; 0.5 = zero correlation; 1.0 = maximal anti-correlation. This is the canonical E–R reporter stored in `DiagReport.phi_er3`.

Error Codes.

Code	Trigger	Recovery
`ER_CORRELATION_DEGENERATE`	Either variance $\sum_i (R_i - \bar{R})^2 = 0$ or $\sum_i (U_i - \bar{U})^2 = 0$	Raise `CREError`; oracle/envi config must be repaired
`MISSING_ENV_SCORES`	Any env ID in `spec.E_dep` missing from either dict	Raise `CREError`
`SCORE_OUT_OF_RANGE`	Any value $\notin [0,1]$ in either dict	Raise `CREError`

Test Standards.

T1. Perfect alignment. $R_i = U_i$ for all i. Assert `phi_er3 == 0.0` ($\pm 10^{-9}$).

T2. Perfect anti-correlation. $R_i = 1 - U_i$ for all i with non-constant scores. Assert `phi_er3 == 1.0` ($\pm 10^{-9}$).

T3. Zero correlation. Construct R, U as independent random draws (seed=42, $n+1 = 10$ envs). Assert $|$`phi_er3`$-0.5| < 0.20$.

T4. Degenerate R. Set all $R_i = 0.5$ (constant). Assert `ER_CORRELATION_DEGENERATE` raised.

T5. Range guarantee. Assert $0.0 \leq$ `phi_er3` ≤ 1.0 for all valid inputs.

T6. Symmetry. Swapping `reward_scores` and `utility_scores` produces the same result (Pearson correlation is symmetric). Assert delta $< 10^{-9}$.

4 Stage II — Module M3: Enhanced Estimators

4.1 Module Overview

M3 computes the three mechanistic estimators $\tilde{\kappa}_{\text{CR}}$, $\tilde{\gamma}_{\text{EC}}$, $\tilde{\delta}_{\text{ER}}$. **Architectural invariant:** M3 outputs are routed *exclusively* to the Stage III Discrepancy Protocol. Writing any M3 output to a canonical slot in `DiagReport` (`phi_cr2`, `phi_ec_bar`, `phi_er3`) raises `CANONICAL_VIOLATION_ERROR` and halts the pipeline.

4.1.1 M3.compute_kappa_cr

Signature.

```
def M3.compute_kappa_cr(
    policy:       DifferentiablePolicy,
    spec:         SpecS,
    n_samples:    int = CFG.n_ref_traj,   # gradient sample count
) -> float
```

Formula (Part I Eq. (19)).

$$\kappa_{CR} = \mathbb{E}_{\pi^*, \tau \sim \mathcal{E}_{\text{tr}}} \Big[-\cos\big(\nabla_\theta J_R(\pi_\theta),\ \nabla_\theta J_C(\pi_\theta)\big) \Big],$$

where $J_C = -\sum_j \mathbb{E}_\pi[\mathbf{1}[(s_t, a_t) \in K_j]]$.

Input Contract. `policy` must expose a `.parameters()` method (PyTorch-compatible or equivalent). `n_samples` gradient trajectories are used for Monte Carlo estimation.
Output Contract. Returns `float` in $[-1, 1]$. $\kappa_{CR} > 0$: reward and safety gradients oppose each other. Stored in `DiagReport.kappa_cr`; never copied to canonical slots.
Error Codes.

Code	Trigger	Recovery
`GRADIENT_UNAVAILABLE`	`policy` has no differentiable parameters	Raise `CREError`; M3 cannot run
`CANONICAL_VIOLATION_ERROR`	Caller writes result to `phi_cr2`	Halt pipeline; raise `CREError`

Test Standards.

T1. Opposing gradients. Construct a synthetic policy where $\nabla J_R = -\nabla J_C$ exactly. Assert `kappa_cr` $\approx$ `1.0`.
T2. Aligned gradients. Construct $\nabla J_R = \nabla J_C$. Assert `kappa_cr` $\approx$ `-1.0`.
T3. Orthogonal gradients. Construct orthogonal gradient vectors. Assert `|kappa_cr| < 0.1`.
T4. Range guarantee. Assert $-1.0 \leq$ `kappa_cr` ≤ 1.0.
T5. Canonical slot protection. Attempt to write `diag_report.phi_cr2 = kappa_cr`. Assert `CANONICAL_VIOLATION_ERROR` raised.

4.1.2 M3.compute_gamma_ec

Signature.

```
def M3.compute_gamma_ec(
    dataset_dep:  list[Trajectory],   # D_{E_dep} rollouts
    spec:         SpecS,
    sim_model:    BisimulationModel,  # phi_sim: S×S -> [0,1]
) -> float
```

Formula (Part I Eq. (20)). $\gamma_{EC} = 1 - \frac{1}{m_{\text{hard}}} \sum_{j:\sigma_j=\text{hard}} \mathbb{E}_{s\sim\mathcal{D}_{\mathcal{E}_{\text{dep}}}}[\max_{s'\in K_j} \phi_{\text{sim}}(s, s')]$.
Input Contract. `sim_model` must implement `.similarity(s1, s2) -> float`, returning values in $[0, 1]$. The model must have been pre-trained contrastively before this call.
Output Contract. Returns `float` in $[0, 1]$. High values indicate poor soft coverage even accounting for similarity to the critical set. Stored in `DiagReport.gamma_ec`.
Error Codes.

Code	Trigger	Recovery
`SIM_MODEL_NOT_FITTED`	`sim_model` has not been trained	Raise `CREError`
`EMPTY_KJ_FOR_GAMMA`	K_j has zero states for some hard C_j	Skip that j; append `KJ_EMPTY_WARN`

Test Standards.

T1. High similarity. All deployment states highly similar to K_j states ($\phi_{\text{sim}} \approx 1$). Assert `gamma_ec` $\approx$ `0.0`.
T2. Low similarity. Deployment states dissimilar from K_j. Assert `gamma_ec` $\approx$ `1.0`.
T3. Range guarantee. Assert $0.0 \leq$ `gamma_ec` ≤ 1.0.

4.1.3 M3.compute_delta_er

Signature.

```
def M3.compute_delta_er(
    policy:       DifferentiablePolicy,
    spec:         SpecS,
    eps_stab:     float = CFG.eps_stab,   # default 1e-6
    n_samples:    int   = CFG.n_ref_traj,
) -> float
```

Formula (Part I Eq. (21)).

$$\delta_{ER} = \max_{i\in\{1,\ldots,n\}} \frac{\|\nabla_\theta R(\tau;\, e_i) - \nabla_\theta R(\tau;\, e_0)\|_2}{\|\nabla_\theta R(\tau;\, e_0)\|_2 + \texttt{eps_stab}}.$$

Output Contract. Returns `float` ≥ 0. Large values indicate the reward gradient direction shifts substantially across deployment environments. Stored in `DiagReport.delta_er`.
Error Codes.

Code	Trigger	Recovery
`GRADIENT_UNAVAILABLE`	policy not differentiable	Raise `CREError`
`SINGLE_ENV_DELTA`	`len(spec.E_dep)==1`	Return `0.0`; append `SINGLE_ENV_WARN`

Test Standards.

T1. Identical environments. $e_i = e_0$ for all i. Assert `delta_er == 0.0`.
T2. Positive value. Environments with shifted dynamics. Assert `delta_er > 0`.
T3. Stability epsilon. Set nominal gradient to zero vector, `eps_stab=1e-6`. Assert no division-by-zero error; result is finite.

5 Stage III — Discrepancy Protocol

5.1 Module Overview

The Discrepancy Protocol runs only when both M2 and M3 outputs are available (`CFG.enhanced_estimators_enabled = True`). It uses module M6 (`LLMg`) exclusively in Case A. The protocol produces no changes to `DiagReport`'s *point estimates*; it may only expand uncertainty intervals (Case A) or append flags (Case B).

5.1.1 DP.compute_discrepancy

Signature.

```
def DP.compute_discrepancy(
    phi_hat:   float,   # canonical primary estimator in [0,1]
    kappa_hat: float,   # enhanced estimator in [0,1]
) -> float
```

Formula (Part I Eq. (22)). $\Delta_d = |\hat{\varphi}_d - \hat{\kappa}_d|$.
Output Contract. Returns `float` in $[0, 1]$. Triggers protocol for dimension d iff result $>$ `CFG.eta_disc` (default 0.15).
Error Codes. None. This function is purely arithmetic; invalid ranges are caught by caller validation.
Test Standards.

T1. $\hat{\varphi} = 0.8$, $\hat{\kappa} = 0.3$: assert $\Delta = 0.5$.
T2. $\hat{\varphi} = \hat{\kappa} = 0.5$: assert $\Delta = 0.0$.
T3. Symmetry: assert $\Delta(\hat{\varphi}, \hat{\kappa}) = \Delta(\hat{\kappa}, \hat{\varphi})$.

5.1.2 DP.case_a_handler

Signature.

```
def DP.case_a_handler(
    dim:         Literal["CR","EC","ER"],
    delta:       float,          # Delta_d from DP.compute_discrepancy
    sigma_d:     float,          # current uncertainty for dim d
    diag_report: DiagReport,     # mutated in place (flags + CI only)
    spec:        SpecS,
    lambda_exp:  float = CFG.lambda_expand,    # default 0.5
    eta_disc:    float = CFG.eta_disc,         # default 0.15
    tau_low:     float = CFG.tau_low,          # default 0.30
    tau_high:    float = CFG.tau_high,         # default 0.70
    llm_backend: str   = "default",
) -> float   # returns updated sigma_d
```

Trigger Condition. Case A fires iff simultaneously: $\hat{\varphi}_d < \tau_{\text{low}}$, $\hat{\kappa}_d > \tau_{\text{high}}$, $\Delta_d > \eta_{\text{disc}}$.
Actions (Part I Invariant 4.1).

1. Expand uncertainty: $\sigma_d \leftarrow \sigma_d \cdot (1 + \lambda \cdot \Delta_d)$ (Part I Eq. (23)).
2. Append `"LATENT_INCONSISTENCY"` to `diag_report.flags`.
3. Call M6 (`LLMg`) to generate a temporal hypothesis (string) appended to `diag_report.discrepancy` as a `DiscrepancyRecord` with `case="A"`.
4. **Do not** modify `diag_report.psi_cre` or any canonical estimator value.

Output Contract. Returns the updated `sigma_d` (expanded value). `diag_report` is mutated in-place (flags and discrepancy list only).
Error Codes.

Code	Trigger
`POINT_ESTIMATE_MUTATED`	Any write to `diag_report.psi_cre`, `phi_cr2`, `phi_ec_bar`, or `phi_er3` inside this func
`LLM_RETRY_EXHAUSTED`	M6 (LLMg) fails after `CFG.llm_retry_limit` attempts

Test Standards.

T1. Uncertainty expansion. Input $\sigma_d = 0.1$, $\Delta_d = 0.40$, $\lambda = 0.5$. Assert returned $\sigma_d = 0.1 \times (1 + 0.5 \times 0.40) = 0.120$.
T2. Flag appended. After call, assert `"LATENT_INCONSISTENCY"` in `diag_report.flags`.
T3. Point estimate immutability. Assert `diag_report.psi_cre` unchanged before and after call.
T4. Trigger condition enforcement. Pass $\hat{\varphi}_d = 0.8$ (not $< \tau_{\text{low}}$). Assert function returns without action (`sigma_d` unchanged, no flag appended).
T5. M6 failure fallback. Mock M6 to raise `LLM_RETRY_EXHAUSTED`. Assert `M6_HYPOTHESIS_UNAVAILABLE` in flags and no pipeline halt.

5.1.3 DP.case_b_handler

Signature.

```
def DP.case_b_handler(
    dim:         Literal["CR","EC","ER"],
    delta:       float,
    diag_report: DiagReport,   # flags appended; no numerical change
```

```
    n_rechk:     int = CFG.n_rechk,   # default 1
    tau_low:     float = CFG.tau_low,
    tau_high:    float = CFG.tau_high,
    eta_disc:    float = CFG.eta_disc,
) -> int   # returns scheduled re-evaluation iteration
```

Trigger Condition. Case B fires iff simultaneously: $\hat{\varphi}_d > \tau_{\text{high}}$, $\hat{\kappa}_d < \tau_{\text{low}}$, $\Delta_d > \eta_{\text{disc}}$.
Actions.

1. Append `"CRITIC_QUALITY_WARN"` to `diag_report.flags`.
2. Canonical primary value $\hat{\varphi}_d$ is held unchanged.
3. Return `n_rechk` as the scheduled re-evaluation iteration.

Error Codes.

Code	Trigger	Recovery
`POINT_ESTIMATE_MUTATED`	Any write to canonical slots inside this function	Halt pipeline

Test Standards.

T1. Flag appended. After call, assert `"CRITIC_QUALITY_WARN"` in flags.
T2. Return value. `n_rechk=2`. Assert return = 2.
T3. No numerical change. Assert all `diag_report` numeric fields unchanged.

5.1.4 DP.run_protocol

Signature.

```
def DP.run_protocol(
    diag_report: DiagReport,   # contains both M2 and M3 values
    spec:        SpecS,
    eta_disc:    float = CFG.eta_disc,
    tau_low:     float = CFG.tau_low,
    tau_high:    float = CFG.tau_high,
    lambda_exp:  float = CFG.lambda_expand,
    n_rechk:     int   = CFG.n_rechk,
) -> DiagReport
```

Logic. For each dimension $d \in \{\text{CR}, \text{EC}, \text{ER}\}$:

1. Retrieve $(\hat{\varphi}_d, \hat{\kappa}_d)$ from `diag_report` per Table 1. If $\hat{\kappa}_d$ is `None` (M3 not run), skip dimension.
2. Compute Δ_d via `DP.compute_discrepancy`.
3. If $\Delta_d \leq \eta_{\text{disc}}$: no-trigger pass-through; write null-discrepancy log entry to audit trail.
4. If Case A condition: call `DP.case_a_handler`.
5. Else if Case B condition: call `DP.case_b_handler`.

Output Contract. Returns the mutated `diag_report` with updated `flags`, `discrepancy`, and CI fields. `psi_cre` and all canonical estimator values are unchanged.
Error Codes. Propagates errors from sub-handlers.
Test Standards.

T1. No-trigger. Set $\Delta_d = 0.10 < 0.15$ for all dims. Assert no flags added; null-discrepancy record written to audit trail.
T2. Case A fires for CR. Set $\hat{\varphi}_{CR} = 0.20$, $\hat{\kappa}_{CR} = 0.80$, $\Delta_{CR} = 0.60$. Assert `LATENT_INCONSISTENCY` in flags and `diag_report.ci_95["phi_cr2"]` is wider than before.

T3. Case B fires for ER. Set $\hat{\varphi}_{ER} = 0.80$, $\hat{\kappa}_{ER} = 0.20$, $\Delta_{ER} = 0.60$. Assert `CRITIC_QUALITY_WARN` in flags.

T4. Missing M3 values. Set `diag_report.kappa_cr = None`. Assert CR dimension is silently skipped (no error).

6 Stage IV — Module M5: Composite Scoring

6.1 Module Overview

M5 is purely deterministic. It applies the monotone transform $f(\cdot)$ and computes the final Ψ_{CRE} score and its bootstrap confidence interval. M5 must be called *after* M2 (and optionally after the Discrepancy Protocol) but *before* M4 (which reads the completed report).

6.1.1 M5.monotone_transform

Signature.

```
def M5.monotone_transform(
    x:          float,
    k:          float = CFG.f_steepness,    # default 8.0
    x0:         float = CFG.f_inflection,   # default 0.5
) -> float
```

Formula (Part I Eq. (27)).

$$f(x) = \frac{\sigma(k(x - x_0)) - \sigma(-kx_0)}{\sigma(k(1 - x_0)) - \sigma(-kx_0)}, \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Input Contract. $x \in [0, 1]$; $k > 0$; $x_0 \in (0, 1)$.
Output Contract. Returns `float` in $[0, 1]$ with $f(0) = 0$ and $f(1) = 1$. Strictly increasing on $(0, 1)$.
Error Codes.

Code	Trigger	Recovery
`TRANSFORM_INPUT_OUT_OF_RANGE`	$x \notin [0, 1]$	Raise `CREError`
`TRANSFORM_K_NON_POSITIVE`	$k \leq 0$	Raise `CREError`
`TRANSFORM_X0_BOUNDARY`	$x_0 \leq 0$ or $x_0 \geq 1$	Raise `CREError`

Test Standards.

T1. $f(0) = 0.0$ ($\pm 10^{-9}$) for default parameters.
T2. $f(1) = 1.0$ ($\pm 10^{-9}$) for default parameters.
T3. $f(0.5) = 0.5$ ($\pm 10^{-9}$) for $x_0 = 0.5$ (symmetry at inflection).
T4. Monotonicity: $f(0.3) < f(0.5) < f(0.7)$.
T5. Out-of-range: $x = 1.1$ raises `TRANSFORM_INPUT_OUT_OF_RANGE`.
T6. Steepness effect: larger k produces value closer to $\{0, 1\}$ at $x = 0.6$: assert $f_{k=20}(0.6) > f_{k=8}(0.6)$.

6.1.2 M5.compute_psi_cre

Signature.

```
def M5.compute_psi_cre(
    phi_cr2:     float,    # canonical C-R reporter in [0,1]
    phi_ec_bar:  float,    # canonical E-C reporter in [0,1]
    phi_er3:     float,    # canonical E-R reporter in [0,1]
```

```
    w_cr:        float = CFG.w_cr,    # default 1/3
    w_ec:        float = CFG.w_ec,    # default 1/3
    w_er:        float = CFG.w_er,    # default 1/3
    f_k:         float = CFG.f_steepness,
    f_x0:        float = CFG.f_inflection,
) -> float
```

Formula (Part I Eq. (26)).

$$\Psi_{\mathrm{CRE}}(\mathcal{S}) = 1 - \Big[w_{CR}\, f(\phi_{CR}^{(2)}) + w_{EC}\, f(1 - \bar{\phi}_{EC}^{(1)}) + w_{ER}\, f(\phi_{ER}^{(3)})\Big].$$

Note: the E–C term uses $f(1 - \phi_{EC}^{(1)})$, not $f(\phi_{EC}^{(1)})$. $\phi_{EC}^{(1)} = 1.0$ (full coverage) contributes $f(0) = 0$ to the penalty.

Input Contract. All three canonical reporters in $[0, 1]$. $w_{CR} + w_{EC} + w_{ER} = 1.0$ (verified at pipeline start; re-checked here in debug mode).

Output Contract. Returns `float` in $[0, 1]$: 1.0 = fully consistent; 0.0 = maximally inconsistent across all three dimensions.

Error Codes.

Code	Trigger
`REPORTER_OUT_OF_RANGE`	Any input $\notin [0, 1]$
`CONFIG_WEIGHT_SUM_ERROR`	$\lvert w_{CR} + w_{EC} + w_{ER} - 1\rvert > 10^{-9}$
`INVERTED_EC_TERM`	Caller passes $1 - \phi_{EC}$ pre-inverted (caught by range check if > 1, but otherwise silent

Test Standards.

T1. Fully consistent. Input $(0.0, 1.0, 0.0)$ for $(\phi_{CR}, \phi_{EC}, \phi_{ER})$. Assert `psi_cre == 1.0` ($\pm 10^{-9}$).

T2. Maximally inconsistent. Input $(1.0, 0.0, 1.0)$. Assert `psi_cre == 0.0` ($\pm 10^{-9}$).

T3. Single dimension inconsistency. Input $(1.0, 1.0, 0.0)$ (perfect C–R and E–C, maximal E–R). Assert `psi_cre` $\approx$ `1 - w_er` (since $f(1) = 1$).

T4. Weight sensitivity. Set $w_{CR} = 0.5$, $w_{EC} = 0.3$, $w_{ER} = 0.2$. Input $(1.0, 0.0, 0.0)$ (only E–C inconsistent). Assert `psi_cre` $\approx$ `1 - 0.3*f(1) = 0.7`.

T5. EC inversion guard. Input $\phi_{EC} = 0.0$ (zero coverage, maximally bad). Assert $f(1 - 0.0) = f(1) = 1$ contributes maximum penalty. Assert `psi_cre < 0.4` (EC dominates with equal weights).

T6. Range guarantee. Sweep 100 random inputs in $[0, 1]^3$. Assert all outputs in $[0, 1]$.

6.1.3 M5.compute_bootstrap_ci

Signature.

```
def M5.compute_bootstrap_ci(
    trajectories:  list[Trajectory],
    spec:          SpecS,
    dim:           Literal["CR","EC","ER","PSI"],
    B:             int   = CFG.b_bootstrap,   # default 1000
    confidence:    float = 0.95,
    seed:          int   = 42,
) -> tuple[float, float]   # (lower, upper) BCa-adjusted
```

Method. Bootstrap B trajectory resamples with replacement. Apply the bias-corrected and accelerated (BCa) method to compute (α_1, α_2) adjusted quantiles. For `dim="PSI"`: re-compute all three canonical reporters on each resample, then apply `M5.compute_psi_cre`.

Input Contract. `confidence` $\in (0,1)$; `B` ≥ 100. Stage III uncertainty expansions (if any) are propagated via first-order error propagation applied *after* bootstrap CI computation.
Output Contract. Returns `(lower, upper)` with `lower` $\leq$ point estimate $\leq$ `upper`. Stored in `DiagReport.ci_95[dim]`.
Error Codes.

Code	Trigger	Recovery
`INSUFFICIENT_BOOTSTRAP_SAMPLES`	$B < 100$	Raise `CREError`
`CI_INVERSION`	`lower > upper` (numerical instability)	Swap; append `CI_INVERSION_WARN` flag

Test Standards.

T1. CI contains point estimate. For any valid input, assert `lower` $\leq$ `point_estimate` $\leq$ `upper`.
T2. CI narrows with more data. Run with 50 vs. 500 trajectories. Assert CI width with 500 trajectories $<$ width with 50.
T3. Reproducibility. Same `seed` produces identical CI bounds.
T4. Small B. `B=50`: assert `INSUFFICIENT_BOOTSTRAP_SAMPLES` raised.

6.1.4 M5.calibrate_thresholds

Signature.

```
def M5.calibrate_thresholds(
    calibration_corpus: list[tuple[SpecS, float]],  # (spec, psi_cre) pairs
    k_detect:  float = CFG.k_detect,  # default 1.5
    k_alarm:   float = CFG.k_alarm,   # default 3.0
) -> tuple[float, float]   # (tau_detect, tau_alarm)
```

Formulae (Part I Eqs. (24)–(25)).

$$\tau_{\text{detect}} = \mu_0 - k_d\,\sigma_0, \qquad \tau_{\text{alarm}} = \mu_0 - k_a\,\sigma_0,$$

where μ_0, σ_0 are mean and std of Ψ_{CRE} on the calibration corpus (assumed to contain consistent specifications).
Input Contract. `calibration_corpus`: ≥ 30 consistent `(spec, psi_cre)` pairs. $k_a > k_d > 0$ (enforced; else `THRESHOLD_ORDER_VIOLATED` raised).
Output Contract. Returns $(\tau_{\text{detect}}, \tau_{\text{alarm}})$ with $\tau_{\text{alarm}} \leq \tau_{\text{detect}}$ (Part I Eq. (6)).
Error Codes.

Code	Trigger	Recovery
`THRESHOLD_ORDER_VIOLATED`	$k_a \leq k_d$	Raise `CREError`
`SMALL_CALIBRATION_CORPUS`	< 30 pairs	Warn; proceed with caveat logged

Test Standards.

T1. Ordering invariant. Assert $\tau_{\text{alarm}} \leq \tau_{\text{detect}}$ always.
T2. Gaussian null, $k_d = 1.5$. Calibrate on 1000 consistent specs with known $\mu_0 = 0.90$, $\sigma_0 = 0.05$. Assert $\tau_{\text{detect}} \approx 0.825$ (± 0.005).
T3. Threshold violation. Pass $k_d = 3.0$, $k_a = 1.5$. Assert `THRESHOLD_ORDER_VIOLATED` raised.

7 Stage IV — Module M4: Semantic Analysis

7.1 Module Overview

M4 (`LLMb`) receives the completed `DiagReport` (after M5) and generates natural-language diagnostic output. **Hard constraint: M4 must not modify any numerical field of `DiagReport`.** A diff guard is applied post-call; violation raises `SEMANTIC_OVERWRITE_ERROR` and the pipeline halts.

7.1.1 M4.generate_failure_hypothesis

Signature.

```
def M4.generate_failure_hypothesis(
    diag_report:  DiagReport,      # read-only; numerical fields must not change
    spec:         SpecS,
    prompt_template: str = PROMPT_BETA,    # Appendix B.2 of Part I
    llm_backend:  str  = "default",
    retry_limit:  int  = CFG.llm_retry_limit,
    max_tokens:   int  = 1024,
) -> tuple[str, list[str]]   # (failure_hypothesis, repair_targets)
```

Input Contract. `diag_report` must have `psi_cre` set (M5 must have run). `diag_report` is passed by reference but must not be mutated inside this function.
Output Contract. Returns `(failure_hypothesis, repair_targets)` where:

- `failure_hypothesis`: a natural-language string mapping numerical findings to causal specification defects.
- `repair_targets`: an ordered list of strings, each naming a specification component to address (e.g. `"R.w_k:speed_bonus"`).

The caller writes these into `diag_report.failure_hypothesis` and `diag_report.repair_targets` after the diff guard passes.
Post-Call Diff Guard. Before writing, the orchestrator computes a field-level diff between the `DiagReport` snapshot taken before the M4 call and the object after. If any numerical field differs by $> 10^{-9}$, raise `SEMANTIC_OVERWRITE_ERROR` and halt.
Error Codes.

Code	Trigger	Recovery
`SEMANTIC_OVERWRITE_ERROR`	M4 modifies a numerical field of `DiagReport`	Halt pipeline; escalate to human
`LLM_RETRY_EXHAUSTED`	All M4 attempts fail	Set `failure_hypothesis=None`; set `repair_targets=[]`; continue
`EMPTY_REPAIR_TARGETS`	LLM returns empty list for `repair_targets`	Warn; M7 will use full operator vocabulary

Test Standards.

T1. Numerical immutability. After call, assert every numerical field of `diag_report` is byte-identical to the pre-call snapshot.
T2. Non-empty output. For a `diag_report` with `psi_cre=0.4`, assert `len(failure_hypothesis) > 0` and `len(repair_targets) >= 1`.
T3. Retry fallback. Mock LLM to exhaust retries. Assert `failure_hypothesis is None` and pipeline continues.
T4. Overwrite guard. Mock M4 to set `diag_report.psi_cre = 0.99` inside the call. Assert `SEMANTIC_OVERWRITE_ERROR` raised.

8 Stage V — Module M7: Repair Generation

8.1 Module Overview

M7 (`LLMd`) generates, filters, and ranks repair proposals. It implements the two-stage ranking protocol of Part I §6. Proposals failing minimality enforcement are discarded before ranking.

8.1.1 M7.compute_spec_edit_distance

Signature.

```
def M7.compute_spec_edit_distance(
    spec:       SpecS,
    spec_prime: SpecS,
) -> float
```

Formula (Part I Eq. (29)).

$$d_{\text{spec}}(\mathcal{S}, \mathcal{S}') = \frac{\sum_{v \in V} w_v \cdot \mathbf{1}[\mathcal{V}(v, \mathcal{S}) \neq \mathcal{V}(v, \mathcal{S}')]}{\sum_{v \in V} w_v},$$

where semantic importance weights are: reward weights $w_v = 2.0$, clip bounds $w_v = 0.5$, constraint severity $w_v = 3.0$, all other fields $w_v = 1.0$.
Input Contract. Both specs must share the same `spec_id` base (i.e. `spec_prime` is derived from `spec`); different IDs raise `UNRELATED_SPECS`. Both must be frozen `SpecS` objects.
Output Contract. Returns `float` in $[0, 1]$. 0.0: identical specs. 1.0: all weighted nodes differ.
Error Codes.

Code	Trigger	Recovery
`UNRELATED_SPECS`	`spec` and `spec_prime` have unrelated `spec_ids`	Raise `CREError`
`EDIT_DISTANCE_NEGATIVE`	Floating-point underflow produces negative value	Clip to 0.0; append warning

Test Standards.

T1. Identity. $d_{\text{spec}}(\mathcal{S}, \mathcal{S}) = 0.0$.
T2. Severity change. Change one constraint from `sigma="soft"` to `"hard"` ($w_v = 3.0$, only node changed). Assert $d_{\text{spec}} = 3.0/\sum_V w_v$ (verify formula).
T3. Range guarantee. Assert $0.0 \leq d_{\text{spec}} \leq 1.0$.
T4. Symmetry. Assert $d(\mathcal{S}, \mathcal{S}') = d(\mathcal{S}', \mathcal{S})$.

8.1.2 M7.generate_repair_proposals

Signature.

```
def M7.generate_repair_proposals(
    diag_report:  DiagReport,
    spec:         SpecS,
    k_proposals:  int  = CFG.k_proposals,   # default 3; generates 2K=6
    prompt_template: str = PROMPT_DELTA,    # Appendix B.4
    llm_backend:  str  = "default",
    retry_limit:  int  = CFG.llm_retry_limit,
    rejection_feedback: str | None = None,  # prepended on retry loops
) -> list[RepairProposal]   # length up to 2*k_proposals
```

Operator Vocabulary $\mathcal{V}_{\mathcal{R}}$. The LLM must choose operators from exactly the following list. Any proposal referencing an operator not in this list is rejected with `UNKNOWN_OPERATOR`.

Space	Operators
Reward	`ReweightTerms`, `InjectPenalty`, `ModifyShaping`, `AdjustClip`
Environment	`ExpandDomainRand`, `InjectCriticalScenes`, `RescaleShiftOp`
Constraint-adjacent	`AdjustThreshold`, `ScheduleCurriculum`, `ReclassifySeverity`

Input Contract. `rejection_feedback`: if non-None (loop iteration ≥ 2), prepend verbatim to the LLM prompt before the `DiagReport` summary. Each proposal must include: (a) the modified `SpecS`; (b) `operator_class` (one of the above); (c) `declared_side_effects` (a dict mapping non-target dims to a description of predicted impact); (d) `semantic_justification` (≤ 200 tokens); (e) `predicted_delta_psi` (LLM's predicted improvement, or `None`).
Output Contract. Returns up to $2K$ proposals (`RepairProposal` objects). If the LLM returns fewer than K proposals after retries, return what is available (minimum 1 required; else raise `NO_PROPOSALS_GENERATED`).
Error Codes.

Code	Trigger	Recovery
`UNKNOWN_OPERATOR`	Proposal references operator outside $\mathcal{V}_{\mathcal{R}}$	Discard that proposal; continue w remainder
`MISSING_SIDE_EFFECTS_DECLARATION`	Proposal lacks `declared_side_effects`	Discard that proposal
`JUSTIFICATION_TOO_LONG`	`semantic_justification` $>$ 200 tokens	Truncate to 200 tokens; appe `JUSTIFICATION_TRUNCATED` flag
`NO_PROPOSALS_GENERATED`	Zero valid proposals after retries	Raise `CREError`; exit repair lo with `HARD_REJECT`
`LLM_RETRY_EXHAUSTED`	All retry attempts failed	Raise `CREError`

Test Standards.

T1. Proposal count. For `k_proposals=3`, assert $1 \leq$ `len(proposals)` ≤ 6.
T2. Operator validation. Mock LLM to return one proposal with operator `"DeleteSpec"` (not in vocabulary). Assert that proposal is silently discarded and remaining valid proposals are returned.
T3. Rejection feedback prepend. Pass `rejection_feedback="C2 violated: safety regression"`. Assert the string appears at the start of the prompt sent to the LLM.
T4. Justification truncation. Mock LLM to return a 300-token justification. Assert `JUSTIFICATION_TRUNCATED` in proposal flags and stored justification is ≤ 200 tokens.
T5. Zero valid proposals. Mock LLM to return only proposals with unknown operators. Assert `NO_PROPOSALS_GENERATED` raised.

8.1.3 M7.filter_by_minimality

Signature.

```
def M7.filter_by_minimality(
    proposals:  list[RepairProposal],
    spec:       SpecS,
    eta_edit:   float = CFG.eta_edit,   # default 0.20
) -> list[RepairProposal]
```

Logic. For each proposal p: compute $d_{\text{spec}}(\mathcal{S}, p.\texttt{spec_prime})$ via `M7.compute_spec_edit_distance`. Retain p iff $d_{\text{spec}} \leq \eta_{\text{edit}}$. Discarded proposals are logged with code `PROPOSAL_REJECTED_MINIMALITY`.

Output Contract. Returns the filtered subset (possibly empty). If all proposals are discarded, return `[]` without raising; the caller (`M7.run_repair_generation`) handles this case.

Error Codes.

Code	Trigger	Recovery
`PROPOSAL_REJECTED_MINIMALITY`	$d_{\text{spec}} > \eta_{\text{edit}}$ for a proposal	Log; discard proposal; continue

Test Standards.

T1. All pass. All proposals with $d_{\text{spec}} = 0.10 < 0.20$. Assert all returned.

T2. All fail. All proposals with $d_{\text{spec}} = 0.25 > 0.20$. Assert empty list returned (no exception).

T3. Boundary. Proposal with $d_{\text{spec}} = 0.20$ (at threshold). Assert retained (non-strict inequality $\leq$).

8.1.4 M7.rank_proposals

Signature.

```
def M7.rank_proposals(
    proposals:       list[RepairProposal],
    spec:            SpecS,
    psi_cre_current: float,
    ranking_mode:    Literal["rollout_first","llm_only"] = CFG.ranking_mode,
    n_rank_episodes: int   = CFG.n_rank_episodes,   # default 50
    eps_rank:        float = CFG.eps_rank,           # default 0.02
    fixed_seed:      int   = 0,
) -> list[RepairProposal]   # sorted descending by effective score
```

Stage 1 — Lightweight Rollout Ranking (skipped if `ranking_mode=="llm_only"`). Evaluate each proposal's `spec_prime` for `n_rank_episodes` episodes on a single fixed-seed environment. Compute $\hat{\Psi}^{\text{rough}}_{CRE}$ for each and store in `proposal.rough_delta_psi`. Sort descending by $\Delta\hat{\Psi}^{\text{rough}}_k = \hat{\Psi}^{\text{rough}}(\mathcal{S}'_k) - \hat{\Psi}^{\text{rough}}(\mathcal{S})$.

Stage 2 — LLM Tiebreaker. When $|\Delta\hat{\Psi}^{\text{rough}}_k - \Delta\hat{\Psi}^{\text{rough}}_{k'}| < \epsilon_{\text{rank}}$, use `predicted_delta_psi` as secondary key (descending). If `predicted_delta_psi` is `None` for a proposal in `"llm_only"` mode, treat as $-\infty$.

Input Contract. `proposals`: non-empty list (caller must ensure after filtering). `psi_cre_current`: the current Ψ_{CRE} value used to compute rough deltas.

Output Contract. Returns proposals sorted descending by effective quality score. The first element is the highest-ranked proposal and is the first submitted to M8.

Error Codes.

Code	Trigger	Recovery
`LLM_ONLY_MODE_SAFETY_CRITICAL`	`ranking_mode=="llm_only"` and `CFG.safety_level=="critical"`	Warn s `"rollout`
`ROUGH_SCORE_UNAVAILABLE`	Rollout for a proposal fails at environment initialization	Set move to l

Test Standards.

T1. Rollout ordering. Three proposals with rough deltas $\{0.10, 0.30, 0.05\}$. Assert order: $[0.30, 0.10, 0.05]$.

T2. Tiebreaker. Two proposals with rough deltas $\{0.20, 0.21\}$ (diff $= 0.01 < \epsilon_{\text{rank}} = 0.02$) and `predicted_delta_psi` $\{0.15, 0.40\}$. Assert proposal with `predicted_delta_psi=0.40` ranked first.

T3. LLM-only mode. Set `ranking_mode="llm_only"`. Assert no rollout environment is invoked. Assert ordering by `predicted_delta_psi` only.

T4. Safety-critical override. Set both `"llm_only"` mode and `safety_level="critical"`. Assert `LLM_ONLY_MODE_SAFETY_CRITICAL` warning raised and mode overridden to `"rollout_first"`.

9 Stage VI — Module M8: Acceptance and Validation

9.1 Module Overview

M8 (`LLMe`) implements the multi-criteria acceptance protocol (Part I §7). A proposal is accepted iff all of C1–C4 pass *and* $s_{\text{sem}} \geq \tau_{\text{sem}}$ and `intent_preserved = True`. On rejection, a `RejectionFeedback` record is produced and prepended to the next M7 call.

9.1.1 M8.evaluate_acceptance_criteria

Signature.

```
def M8.evaluate_acceptance_criteria(
    spec:        SpecS,
    spec_prime:  SpecS,
    diag_orig:   DiagReport,   # computed on spec
    diag_prime:  DiagReport,   # computed on spec_prime (full M2 run)
    eta_edit:    float = CFG.eta_edit,    # default 0.20
    eps_perf:    float = CFG.eps_perf,   # default 0.05
) -> AcceptanceVerdict
```

Criteria (Part I Eqs. (30)–(33)).

$$\begin{aligned}
&\text{C1 (Diagnostic):} \quad \Psi_{\text{CRE}}(\mathcal{S}') > \Psi_{\text{CRE}}(\mathcal{S}),\\
&\text{C2 (Safety):} \quad \max_j P_{\pi^*(\mathcal{S}')}[(s,a) \in K_j] \leq \max_j P_{\pi^*(\mathcal{S})}[(s,a) \in K_j],\\
&\text{C3 (Utility):} \quad U_{\text{nom}}(\mathcal{S}') \geq U_{\text{nom}}(\mathcal{S}) - \epsilon_{\text{perf}},\\
&\text{C4 (Minimality):} \quad d_{\text{spec}}(\mathcal{S}, \mathcal{S}') \leq \eta_{\text{edit}}.
\end{aligned}$$

Output Contract. Returns `AcceptanceVerdict` with all fields set. `accepted = True` iff all of C1, C2, C3, C4 pass (M8 semantic check is evaluated separately in `M8.verify_semantic_consistency`).

Error Codes.

Code	Trigger	Recovery
`PROPOSAL_REJECTED_DIAGNOSTIC`	C1 fails: $\Psi_{\text{CRE}}(\mathcal{S}') \leq \Psi_{\text{CRE}}(\mathcal{S})$	Record in r try next pro
`PROPOSAL_REJECTED_SAFETY`	C2 fails: safety regression	Record; try
`PROPOSAL_REJECTED_UTILITY`	C3 fails: utility drops by $> \epsilon_{\text{perf}}$	Record; try
`PROPOSAL_REJECTED_MINIMALITY`	C4 fails (should have been caught by `M7.filter_by_minimality`)	Record; th guard

Test Standards.

T1. All pass. Set: $\Psi'_{\text{CRE}} = 0.80 > \Psi_{\text{CRE}} = 0.70$ (C1), safety non-regressed (C2), utility drop $= 0.02 < 0.05$ (C3), $d = 0.10 < 0.20$ (C4). Assert `accepted=True` and all `c*_pass=True`.

T2. C1 failure. $\Psi'_{\text{CRE}} = 0.65 < \Psi_{\text{CRE}} = 0.70$. Assert `c1_pass=False`, `accepted=False`, `rejection_feedback` contains `"PROPOSAL_REJECTED_DIAGNOSTIC"`.

T3. C2 failure. Safety violation probability increases. Assert `c2_pass=False`, `accepted=False`.

T4. C3 boundary. Utility drop $= \epsilon_{\text{perf}} = 0.05$ exactly. Assert `c3_pass=True` (non-strict: $\geq U - \epsilon$).

T5. Multiple failures. C1 and C3 both fail. Assert `accepted=False`; assert both codes appear in `rejection_feedback`.

9.1.2 M8.verify_semantic_consistency

Signature.

```
def M8.verify_semantic_consistency(
    nl_orig:      NLInput,        # original natural-language input
    spec_prime:   SpecS,          # proposed repaired spec
    prompt_template: str = PROMPT_EPSILON,   # Appendix B.5
    tau_sem:      float = CFG.tau_sem,       # default 0.80
    llm_backend:  str   = "default",
    retry_limit:  int   = CFG.llm_retry_limit,
) -> tuple[float, bool]   # (s_sem, intent_preserved)
```

Method. M8 (`LLMe`) receives `nl_orig` and the de-parsed `nl_prime` (obtained by inverse-mapping `spec_prime` back to natural language via a deterministic serialiser). The LLM returns `s_sem` $\in [0, 1]$ and `intent_preserved` (`bool`). Acceptance additionally requires `s_sem` $\geq$ `tau_sem` AND `intent_preserved = True`.

Output Contract. Returns `(s_sem, intent_preserved)`. The final acceptance decision is: `accepted = (all C1-C4 pass) AND (s_sem >= tau_sem) AND intent_preserved`.

Error Codes.

Code	Trigger	Recovery
`SEM_SCORE_OUT_OF_RANGE`	LLM returns `s_sem` $\notin [0, 1]$	Clip to $[0, 1]$; append warning
`LLM_RETRY_EXHAUSTED`	M8 call fails after retries	Set `s_sem=0` `intent_preserved=False`; posal is rejected
`DEPARSING_FAILURE`	`spec_prime` cannot be serialized to natural language	Raise `CREError`

Test Standards.

T1. High semantic consistency. `spec_prime` is a minor reward reweighting. Assert $s_{\text{sem}} \geq 0.90$ and `intent_preserved=True`.

T2. Low consistency. `spec_prime` completely replaces all reward terms. Assert $s_{\text{sem}} < \tau_{\text{sem}} = 0.80$ and `intent_preserved=False`.

T3. Threshold boundary. Mock $s_{\text{sem}} = 0.80$ exactly. Assert proposal passes semantic check (`>= tau_sem`, non-strict).

T4. Retry exhaustion. Mock M8 to fail all retries. Assert `(0.0, False)` returned and proposal rejected.

9.1.3 M8.run_acceptance_loop

Signature.

```
def M8.run_acceptance_loop(
    proposals:       list[RepairProposal],   # ranked by M7.rank_proposals
    spec:            SpecS,
    diag_orig:       DiagReport,
    nl_orig:         NLInput,
    n_max:           int   = CFG.n_max_iterations,   # default 5
    eta_edit:        float = CFG.eta_edit,
    eps_perf:        float = CFG.eps_perf,
    tau_sem:         float = CFG.tau_sem,
    generate_fn:     Callable = M7.generate_repair_proposals,
    rank_fn:         Callable = M7.rank_proposals,
    filter_fn:       Callable = M7.filter_by_minimality,
) -> tuple[SpecS | None, AcceptanceVerdict, str]
  # returns (accepted_spec, verdict, exit_code)
  # exit_code in {"ACCEPT", "HARD_REJECT"}
```

Logic.

1. For each proposal in `proposals` (by rank order):
   (a) Run full M2 on `proposal.spec_prime` to get `diag_prime`.
   (b) Call `M8.evaluate_acceptance_criteria`.
   (c) Call `M8.verify_semantic_consistency`.
   (d) If both pass: return `(proposal.spec_prime, verdict, "ACCEPT")`.
   (e) Else: accumulate rejection feedback.
2. If all proposals exhausted in current iteration, call `generate_fn` with accumulated `rejection_feedback` prepended, then `filter_fn`, then `rank_fn`. Increment iteration counter.
3. If iteration counter > `n_max`: return `(None, best_verdict, "HARD_REJECT")` where `best_spec = argmax`$_k \Psi_{\mathrm{CRE}}(\mathcal{S}'_k)$.

Output Contract.

- On `"ACCEPT"`: `accepted_spec` is the validated `SpecS'` with `spec.version` incremented.
- On `"HARD_REJECT"`: `accepted_spec` is `None`; a human-escalation alert is written to the audit trail containing the best-seen proposal.

Error Codes.

Code	Trigger	Recovery
`HARD_REJECT`	Loop exceeds `n_max` iterations without acceptance	Log best proposal; write escala alert; return `None`
`NO_PROPOSALS_AFTER_FILTER`	All proposals in an iteration fail minimality filter	Regenerate with increased feedl count as iteration

Test Standards.

T1. Accept on first proposal. Provide one proposal satisfying all criteria. Assert exit code `"ACCEPT"` after one M2 evaluation.

T2. Accept on second proposal. First proposal fails C2; second passes all criteria. Assert exit code `"ACCEPT"`; assert `rejection_feedback` in second M7 call contains C2 failure.

T3. Hard reject. Set `n_max=2`; mock M7 to always produce proposals failing C1. Assert `"HARD_REJECT"` returned after 2 iterations; assert `accepted_spec is None`.

T4. Best-seen escalation. On `HARD_REJECT`, assert audit trail contains the proposal with highest $\Psi_{\mathrm{CRE}}(\mathcal{S}')$ seen across all iterations.

T5. Version increment. On `"ACCEPT"`, assert `accepted_spec.version == spec.version + 1`.

10 Pipeline Orchestrator

10.1 Module Overview

The orchestrator is the top-level entry point. It sequences all modules in the correct order, manages the audit trail, enforces the weight constraint, and handles early exits.

10.1.1 PO.run_cre_pipeline

Signature.

```
def PO.run_cre_pipeline(
    nl:              NLInput,
    cfg:             CREConfig = CFG,
    audit_trail:     AuditTrail | None = None,
    rollout_policy:  Policy | None = None,    # pi_star approximation
) -> PipelineResult
```

PipelineResult schema.

```
@dataclass
class PipelineResult:
    exit_code:      Literal["ACCEPT","HARD_REJECT","PRECHECK_FAIL",
                            "CONSISTENT"]
    spec_original:  SpecS
    spec_repaired:  SpecS | None   # None on HARD_REJECT or CONSISTENT
    diag_report:    DiagReport
    n_iterations:   int
    audit_trail:    AuditTrail
    human_alert:    str | None     # populated on HARD_REJECT
```

Execution Sequence.

1. Validate `CFG` (weight sum, all positive).
2. `M1.parse_nl_input` → `(spec, flags)`.
3. `M1.detect_and_escalate_ambiguities`.
4. `M1.run_symbolic_precheck`. On fatal: exit `"PRECHECK_FAIL"`.
5. `M2.run_all`: compute all primary estimators.
6. If `CFG.enhanced_estimators_enabled`: `M3.run_all`.
7. If M3 ran: `DP.run_protocol`.
8. `M5.compute_psi_cre` + `M5.compute_bootstrap_ci`.
9. If `psi_cre >= cfg.tau_alarm_psi`: exit `"CONSISTENT"` (no repair required).
10. `M4.generate_failure_hypothesis`.
11. `M7.generate_repair_proposals` → `M7.filter_by_minimality` → `M7.rank_proposals`.
12. `M8.run_acceptance_loop` → `exit_code` ∈ `{ACCEPT, HARD_REJECT}`.
13. Finalize audit trail; return `PipelineResult`.

Output Contract. `exit_code = "CONSISTENT"`: $\Psi_{\text{CRE}} \geq \tau_{\text{alarm}}$; no repair attempted. `exit_code = "ACCEPT"`: `spec_repaired` is the validated repaired specification. `exit_code = "HARD_REJECT"`: repair loop exhausted; `spec_repaired=None`; `human_alert` is populated. `exit_code = "PRECHECK_FAIL"`: M1 pre-check halted pipeline.

Error Codes. Propagates all sub-module errors. Top-level uncaught `CREErrors` are re-raised with full stack trace attached to the audit trail.

Test Standards.

T1. Consistent spec. Provide a fully consistent spec (all reporters ≈ 0 except `phi_ec_bar=1.0`). Assert `exit_code="CONSISTENT"` and no M7/M8 calls made.

T2. End-to-end accept. Provide a spec with known C–R inconsistency and a mock M7 that returns a valid reweighting proposal. Assert `exit_code="ACCEPT"` and `result.spec_repaired.version==1`.

T3. End-to-end hard reject. Set `n_max=1`; mock all M7 proposals to fail C1. Assert `exit_code="HARD_REJECT"` and `human_alert` non-empty.

T4. Precheck failure. Provide a spec where all reward support lies inside K_j. Assert `exit_code="PRECHECK_FAIL"`.

T5. Audit trail completeness. After any run, assert audit trail contains records for every module that executed (M1, M2, M5 always; M3, DP conditionally).

T6. Configuration validation. Set `w_cr=0.5`, `w_ec=0.5`, `w_er=0.5` (sum = 1.5). Assert `CONFIG_WEIGHT_SUM_ERROR` raised before any module runs.

10.1.2 PO.audit_trail_schema

Every module appends a typed record to the `AuditTrail` at completion. The schema is:

```
@dataclass
class AuditRecord:
    record_id:    str           # UUID
    timestamp:    str           # ISO 8601 UTC
    module:       str           # e.g. "M1", "M2.compute_phi_cr2"
    stage:        int           # 1-6
    inputs_hash:  str           # SHA-256 of serialized inputs
    outputs_hash: str           # SHA-256 of serialized outputs
    flags:        list[str]     # error/warning codes emitted
    duration_ms:  float         # wall-clock time
    iteration:    int           # repair loop iteration (0 if pre-loop)
    notes:        str | None    # free-form annotation

@dataclass
class AuditTrail:
    pipeline_id:  str
    records:      list[AuditRecord]

    def append(self, record: AuditRecord) -> None: ...
    def get_by_module(self, module: str) -> list[AuditRecord]: ...
    def to_json(self) -> str: ...
    def from_json(cls, s: str) -> "AuditTrail": ...
```

Invariants. Records are append-only; no record may be modified or deleted after insertion. `inputs_hash` and `outputs_hash` enable post-hoc reproducibility verification.

Test Standards.

T1. Assert `to_json(from_json(trail.to_json())) == trail.to_json()` (round-trip serialization).

T2. Assert record timestamps are monotonically non-decreasing.

T3. Assert `get_by_module("M2.compute_phi_cr2")` returns exactly the records for that function.

11 Error Code Registry

Table 5 provides the complete error code registry. All codes are string constants defined in `cre.errors` and accessible as `CREErrorCode.CODE_NAME`. Codes marked **[HALT]** termi-

nate the pipeline immediately; codes marked **[WARN]** are non-fatal and appended to `DiagReport.flags` or `AuditRecord.flags`.

Table 5: Complete CRE error code registry.

Code	Severity	Module	Description
`SPEC_PARSE_FAILURE`	HALT	M1	LLM failed to produce valid JSON spec after all retries
`NULL_REWARD`	HALT	M1	Reward DAG has no nodes after null-fill
`EMPTY_CONSTRAINT_SET`	WARN	M1	No constraints parsed; downstream runs constraint-free
`EMPTY_ENV_SET`	HALT	M1	No deployment environments; e_0 required
`AMBIGUITY_UNRESOLVABLE`	HALT	M1	High-impact ambiguity flag cannot be auto-resolved
`PRECHECK_TYPE_MISMATCH`	HALT	M1	Incompatible state/action subspaces
`PRECHECK_DOMAIN_TRIVIAL`	HALT	M1	Entire state space is violating; trivial repair exists
`PRECHECK_COVERAGE_INSUFFICIENT`	HALT	M1	Fewer scene families than hard constraints
`PRECHECK_SOFT_NO_TOLERANCE`	WARN	M1	Soft constraint with `zeta=None`
`DEGENERATE_CR1_VAR`	WARN	M2	$\sigma_R = 0$ or $\sigma_C = 0$; `phi_cr1` set to `None`
`BOUNDARY_BASELINE_DEGENERATE`	HALT	M2	π_{unif} has zero boundary clearance
`ER_CORRELATION_DEGENERATE`	HALT	M2	Zero cross-environment variance in R or U
`NO_HARD_CONSTRAINTS`	HALT	M2	`phi_cr2` called with no hard constraints
`WRONG_SEVERITY`	HALT	M2	`compute_phi_ec_j` called with non-hard constraint
`EMPTY_KJ_ESTIMATE`	WARN	M2	MC sampling yields 0 points in K_j
`EMPTY_HARD_CONSTRAINTS`	HALT	M2	`compute_phi_ec_bar` called with empty list
`INSUFFICIENT_TRAJECTORIES`	HALT	M2	Trajectory list below minimum (30)
`MISSING_NOMINAL_UTILITY`	HALT	M2	Nominal env score missing from utility dict
`MISSING_ENV_SCORES`	HALT	M2	An env ID in `spec.E_dep` not in score dict
`SCORE_OUT_OF_RANGE`	HALT	M2	A reward or utility score outside $[0, 1]$

(continued from previous page)

Code	Severity	Module	Description
`SINGLE_ENV_WARN`	WARN	M2	Only one environment; gap metrics return 0
`SINGLE_CONSTRAINT_DIVERSITY`	WARN	M2	$m = 1$; diversity returns 1.0 by convention
`ZERO_CONSTRAINT_ACTIVATIONS`	WARN	M2	All $p_j = 0$; diversity returns 0.0
`KJ_EMPTY_WARN`	WARN	M2/M3	K_j contains no sampled states
`REWARD_OUT_OF_RANGE`	HALT	M2	Normalized reward score outside $[0, 1]$
`CANONICAL_VIOLATION_ERROR`	HALT	M2/M3/DP	Enhanced or supplementary value written to canonical slot
`GRADIENT_UNAVAILABLE`	HALT	M3	Policy has no differentiable parameters
`SIM_MODEL_NOT_FITTED`	HALT	M3	Bisimulation model not yet trained
`LATENT_INCONSISTENCY`	WARN	DP	Case A triggered; uncertainty interval expanded
`CRITIC_QUALITY_WARN`	WARN	DP	Case B triggered; critic/gradient estimator may be unreliable
`POINT_ESTIMATE_MUTATED`	HALT	DP	Case handler modified a canonical estimator value
`M6_HYPOTHESIS_UNAVAILABLE`	WARN	DP	M6 (LLMg) exhausted retries; temporal hypothesis skipped
`SEMANTIC_OVERWRITE_ERROR`	HALT	M4	M4 modified a numerical field of `DiagReport`
`EMPTY_REPAIR_TARGETS`	WARN	M4	LLM returned empty repair-target list
`TRANSFORM_INPUT_OUT_OF_RANGE`	HALT	M5	$x \notin [0, 1]$ passed to $f(\cdot)$
`TRANSFORM_K_NON_POSITIVE`	HALT	M5	Steepness $k \leq 0$
`TRANSFORM_X0_BOUNDARY`	HALT	M5	Inflection $x_0 \notin (0, 1)$
`REPORTER_OUT_OF_RANGE`	HALT	M5	Canonical reporter value outside $[0, 1]$
`CONFIG_WEIGHT_SUM_ERROR`	HALT	M5/PO	$\lvert w_{CR} + w_{EC} + w_{ER} - 1 \rvert > 10^{-9}$
`INSUFFICIENT_BOOTSTRAP_SAMPLES`	HALT	M5	$B < 100$
`CI_INVERSION_WARN`	WARN	M5	Bootstrap lower > upper; values swapped
`THRESHOLD_ORDER_VIOLATED`	HALT	M5	$k_a \leq k_d$; ordering constraint violated
`SMALL_CALIBRATION_CORPUS`	WARN	M5	Calibration corpus < 30 consistent specs
`INVALID_AGGREGATION_MODE`	HALT	M5	`ec_aggregation_mode` not in `{"mean","min"}`

(continued from previous page)

Code	Severity	Module	Description
UNRELATED_SPECS	HALT	M7	Specs have unrelated spec_ids
EDIT_DISTANCE_NEGATIVE	WARN	M7	Underflow; value clipped to 0
UNKNOWN_OPERATOR	WARN	M7	Proposal references operator outside $\mathcal{V}_{\mathcal{R}}$; proposal discarded
MISSING_SIDE_EFFECTS_DECLARATION	WARN	M7	Proposal lacks declared_side_effects; discarded
JUSTIFICATION_TRUNCATED	WARN	M7	Justification > 200 tokens; truncated
NO_PROPOSALS_GENERATED	HALT	M7	Zero valid proposals after all retries
PROPOSAL_REJECTED_MINIMALITY	WARN	M7/M8	$d_{\text{spec}} > \eta_{\text{edit}}$; proposal discarded
LLM_ONLY_MODE_SAFETY_CRITICAL	WARN	M7	LLM-only ranking in safety-critical mode
ROUGH_SCORE_UNAVAILABLE	WARN	M7	Rollout failed; proposal ranked last
PROPOSAL_REJECTED_DIAGNOSTIC	WARN	M8	C1 fails: $\Psi'_{\text{CRE}} \leq \Psi_{\text{CRE}}$
PROPOSAL_REJECTED_SAFETY	WARN	M8	C2 fails: safety regression
PROPOSAL_REJECTED_UTILITY	WARN	M8	C3 fails: utility drop > ϵ_{perf}
SEM_SCORE_OUT_OF_RANGE	WARN	M8	LLM s_sem outside $[0, 1]$; clipped
DEPARSING_FAILURE	HALT	M8	spec_prime cannot be serialized to NL
HARD_REJECT	WARN	M8	Repair loop exhausted $N_{\max}$ iterations
NO_PROPOSALS_AFTER_FILTER	WARN	M8	All proposals in an iteration removed by minimality filter
LLM_RETRY_EXHAUSTED	HALT	All LLM	All retries exhausted
PRECHECK_FATAL	HALT	M1	Generic pre-check fatal wrapper

12 Integration Test Suite

12.1 Module Unit Tests

Each module must pass its own test battery (defined in each

12.1.1

above) before integration testing begins. The minimum requirements per module are listed in Table 6.

Table 6: Minimum unit test counts per module.

Module	Functions Tested	Min Tests	Coverage Target
M1	3	13	100% branch
M2	10	38	100% branch
M3	3	9	95% branch
DP	4	14	100% branch
M5	4	18	100% branch
M4	1	4	90% (LLM-dependent)
M7	4	16	95% branch
M8	3	12	95% branch
PO	2	9	90% branch
Total	**34**	**133**	—

12.1.2 Numerical Regression Suite

Signature.

```
def test_numerical_regression(
    fixture_path: str = "tests/fixtures/regression_v3.json"
) -> None
```

Fixture format. Each fixture entry is a JSON object:

```
{
  "fixture_id": "NR-001",
  "inputs": { "phi_cr2": 0.7, "phi_ec_bar": 0.5, "phi_er3": 0.3 },
  "expected_psi_cre": 0.5814,  # pre-computed with k=8, x0=0.5, equal weights
  "tolerance": 1e-4
}
```

Test Standards.

T1. NR-001. $(\phi_{CR}, \phi_{EC}, \phi_{ER}) = (0.7, 0.5, 0.3)$, equal weights, default f params. $1-\phi_{EC} = 0.5$; compute $f(0.7) + f(0.5) + f(0.3)$ and apply formula. Assert $|\Psi_{\text{CRE}} - \texttt{expected}| < 10^{-4}$.

T2. NR-002. All reporters zero: $(0.0, 1.0, 0.0)$. Assert $\Psi_{\text{CRE}} = 1.0$ exactly.

T3. NR-003. All reporters maximal: $(1.0, 0.0, 1.0)$. Assert $\Psi_{\text{CRE}} = 0.0$ exactly.

T4. NR-004. $(0.5, 0.5, 0.5)$ with $w_{CR} = 0.5$, $w_{EC} = 0.3$, $w_{ER} = 0.2$. Assert within 10^{-4} of pre-computed reference.

T5. NR-005. Sweep $11^3 = 1331$ grid points over $[0, 1]^3$ with step 0.1. Assert all $\Psi_{\text{CRE}} \in [0, 1]$ and monotone increasing as each reporter moves toward the consistent pole.

12.1.3 End-to-End Pipeline Tests

Signature.

```
def test_e2e_pipeline(
    scenario_path: str = "tests/e2e/scenarios/",
    mock_llm:      bool = True,   # use deterministic mock LLMs
) -> None
```

Required Scenarios.

E1. Consistent spec. A spec with all reporters in the acceptable range. Assert `exit_code="CONSISTENT"` with no M7/M8 invocations.

E2. Class I C–R inconsistency, single repair. A spec with $\phi^{(2)}_{CR} > 0.70$ due to reward-boundary conflict. Mock M7 to return a valid `InjectPenalty` proposal on iteration 1. Assert `exit_code="ACCEPT"`, `n_iterations=1`, `spec_repaired.C` unchanged (reward-space fix only).

E3. Class II E–C inconsistency, multi-iteration repair. A spec with $\bar{\phi}^{(1)}_{EC} < 0.30$ (poor coverage). Mock M7 to fail C2 on iteration 1, succeed with `InjectCriticalScenes` on iteration 2. Assert `exit_code="ACCEPT"`, `n_iterations=2`.

E4. Class III E–R inconsistency with Discrepancy Protocol. A spec with $\phi^{(3)}_{ER} > 0.60$ and $\delta_{ER} > \eta_{\text{disc}}$ (Case A for E–R dimension). Assert `LATENT_INCONSISTENCY` in `diag_report.flags` and CI for `phi_er3` is wider than without Case A. Assert `exit_code="ACCEPT"` after valid repair.

E5. Hard reject scenario. Set `n_max=2`; mock M7 to always produce proposals failing C1. Assert `exit_code="HARD_REJECT"`, `human_alert` non-None, audit trail contains exactly 2 repair iteration records.

E6. Precheck failure. Provide a spec with $\text{support}(R) \subseteq \bigcup_j K_j$. Assert `exit_code="PRECHECK_FAIL"` with no M2 calls.

E7. LLM retry resilience. Mock M1 LLM to fail twice then succeed on attempt 3. Assert `exit_code` is not `"PRECHECK_FAIL"` and pipeline completes normally.

E8. Safety-critical aggregation mode. Set `safety_level="critical"` in config. Assert `ec_aggregation_mode` is automatically set to `"min"` without explicit configuration.

E9. Audit trail completeness. Run any scenario to completion. Assert audit trail contains a record for every module that executed, with monotonically increasing timestamps and non-empty `outputs_hash`.

12.1.4 Termination Rate Benchmark

This benchmark validates the empirical termination rates of the repair loop against the bound in Part I Theorem 8.7.

Signature.

```
def benchmark_termination_rate(
    n_trials:      int   = 500,
    p_min_assumed: float = 0.05,
    delta:         float = 0.05,
    seed:          int   = 0,
) -> dict[str, float]
```

Test Standards.

T1. Theoretical bound check. With `p_min=0.05` and $\delta = 0.05$, the bound gives $N \leq \lceil \log(0.05)/\log(0.95) \rceil = 58$ iterations. Assert that across 500 trials, $\geq 95\%$ of runs find an acceptable proposal within 58 iterations.

T2. Mean iterations. Assert mean iterations across trials $\leq N_{\max} = 5$ on the benchmark task suite (where $p_{\min}$ is approximately met by the LLM proposal distribution).

T3. Hard reject rate. Assert hard reject rate $< 10\%$ on the benchmark suite.